

Enterprise Message API Java Edition 3.10.1.L1

ENTERPRISE MESSAGE API CONFIGURATION GUIDE

Document Version: 3.10.1.L1
Date of issue: September 2026
Document ID: EMAJ3101L1CG.260



LSEG DATA &
ANALYTICS

© **LSEG 2016 - 2026**. All rights reserved.

Republication or redistribution of LSEG Data & Analytics content, including by framing or similar means, is prohibited without the prior written consent of LSEG Data & Analytics. 'LSEG Data & Analytics' and the LSEG Data & Analytics logo are registered trademarks and trademarks of LSEG Data & Analytics.

Any software, including but not limited to: the code, screen, structure, sequence, and organization thereof, and its documentation are protected by national copyright laws and international treaty provisions. This manual is subject to U.S. and other national export regulations.

LSEG Data & Analytics, by publishing this document, does not guarantee that any information contained herein is and will remain accurate or that use of the information will ensure correct and faultless operation of the relevant service or equipment. LSEG Data & Analytics, its agents, and its employees, shall not be held liable to or through any user for any loss or damage whatsoever resulting from reliance on the information contained herein.

Contents

1	Introduction	1
1.1	About this Manual	1
1.2	Audience	1
1.3	Acronyms and Abbreviations	1
1.4	References	2
1.5	Documentation Feedback	3
1.6	Document Conventions	3
1.6.1	<i>Typographic</i>	3
1.6.2	<i>Field and Text Values</i>	3
1.6.3	<i>Boolean Values</i>	4
2	Enterprise Message API Configuration Overview	5
2.1	About Message API Configuration	5
2.2	Parameter Overview	5
2.3	Default Behaviors	5
3	Configuration Groups	6
3.1	Consumer Group	6
3.1.1	<i>Generic XML Schema for ConsumerGroup</i>	6
3.1.2	<i>Setting a Default Consumer</i>	6
3.1.3	<i>Configuring Consumers in a ConsumerGroup</i>	7
3.1.4	<i>Consumer Entry Parameters</i>	7
3.2	IProvider Group	14
3.2.1	<i>Generic XML Schema for Provider Group</i>	14
3.2.2	<i>Setting a Default IProvider</i>	14
3.2.3	<i>Configuring an IProvider in an IProviderGroup</i>	15
3.2.4	<i>IProvider Entry Parameters</i>	15
3.3	NiProvider Group	20
3.3.1	<i>Generic XML Schema for NiProvider Group</i>	20
3.3.2	<i>Setting a Default NiProvider</i>	20
3.3.3	<i>Configuring an NiProvider in an NiProviderGroup</i>	21
3.3.4	<i>NiProvider Entry Parameters</i>	21
3.4	Channel Group	26
3.4.1	<i>Generic XML Schema for ChannelGroup</i>	26
3.4.2	<i>Universal Channel Entry Parameters</i>	27
3.4.3	<i>EMA Channel Connection Types</i>	29
3.4.4	<i>Parameters for Use with Channel Type: RSSL_SOCKET</i>	29
3.4.5	<i>Parameters for Use with Channel Types: RSSL_HTTP</i>	31
3.4.6	<i>Parameters for Use with Channel Types: RSSL_WEBSOCKET</i>	32
3.4.7	<i>Parameters for Use with Channel Types: RSSL_ENCRYPTED</i>	32
3.4.8	<i>Example XML Schema for Configuring ChannelSet</i>	33
3.4.9	<i>Example ChannelSet XML Configuration</i>	33
3.4.10	<i>Example Programmatic Configuration for ChannelSet</i>	34
3.5	Warm Standby Channel Group	35
3.5.1	<i>Generic XML Schema for WarmStandbyGroup</i>	35
3.5.2	<i>Warm Standby Channel Entry Parameters</i>	35
3.6	Warm Standby ServerInfo Group	36
3.6.1	<i>Generic XML Schema for WarmStandbyServerInfoGroup</i>	36
3.6.2	<i>Warm Standby ServerInfo Entry Parameters</i>	37
3.7	SessionChannel Group	37
3.7.1	<i>Generic XML Schema for SessionChannelGroup</i>	37

3.7.2	SessionChannel Entry Parameters.....	37
3.8	Server Group.....	40
3.8.1	Generic XML Schema for ServerGroup.....	40
3.8.2	Server Entry Parameters	40
3.8.3	EMA Server Connection Types	43
3.8.4	Parameters for Use with ServerType RSSL_WEBSOCKET	43
3.9	Dictionary Group	44
3.9.1	Generic XML Schema for DictionaryGroup	44
3.9.2	Dictionary Entry Parameters.....	45
3.10	Directory Group.....	45
3.10.1	Generic XML Schema for Directory Entry.....	45
3.10.2	Setting Default Directory.....	46
3.10.3	Configuring a Directory in a DirectoryGroup.....	46
3.10.4	Service Entry Parameters.....	47
3.10.5	InfoFilter Entry Parameters.....	47
3.10.6	StateFilter Entry Parameters	50
3.10.7	Status Entry Parameters.....	50
3.10.8	Setting Directory with Multiple Dictionaries Provided for IProvider.....	50
3.11	Global Configuration	53
3.11.1	Generic XML Schema and Programmatic Configuration for GlobalConfig.....	53
3.11.2	Global Parameters.....	54
4	Enterprise Message API Configuration Processing	56
4.1	Overview and Configuration Precedence.....	56
4.2	Default Configuration	56
4.2.1	Default Consumer Configuration	56
4.2.2	Default Provider Configurations.....	57
4.3	Processing EMA's XML Configuration File	58
4.3.1	Reading the Configuration File	58
4.3.2	Use of the Correct Order in the XML Schema.....	59
4.3.3	Processing the Consumer "Name".....	60
4.3.4	Processing the Provider "Name".....	60
4.4	Configuring Enterprise Message API Using Method Calls.....	61
4.4.1	EMA Configuration Method Calls.....	61
4.4.2	Using the host() Function: How Host and Port Parameters are Processed.....	67
4.4.3	Service Discovery Configuration Using Function Calls.....	67
4.5	Helper Method Calls.....	68
4.5.1	OmmConsumer Helper Method Calls.....	68
4.6	Programmatic Configuration	69
4.6.1	OMM Data Structure.....	69
4.6.2	Creating a Programmatic Configuration for a Consumer.....	70
4.6.3	Example: Programmatic Configuration of the Consumer	71
4.6.4	Creating a Programmatic Configuration for a Provider.....	73
4.6.5	Example: Programmatic Configuration of a Provider.....	74

1 Introduction

1.1 About this Manual

This document is authored by Enterprise Message API architects and programmers. Several of its authors have designed, developed, and maintained the Enterprise Message API product and other LSEG products which leverage it. As such, this document is concise and addresses realistic scenarios and use cases.

This guide documents the functionality and capabilities of the Enterprise Message API Java Edition . The Enterprise Message API can also connect to and leverage many different LSEG and customer components. If you want the Enterprise Message API to interact with other components, consult that specific component's documentation to determine the best way to configure for optimal interaction.

This document explains the configuration parameters for the Enterprise Messaging API (simply called the Message API). Message API configuration is specified first via compiled-in configuration values, then via an optional user-provided XML configuration file, and finally via programmatic changes introduced via the software.

Configuration works in the same fashion across all platforms.

1.2 Audience

This manual provides information that aids software developers and local site administrators in understanding Enterprise Message API configuration parameters. You can obtain further information from the *Enterprise Message Java Edition API Developer's Guide*.

1.3 Acronyms and Abbreviations

ACRONYM / TERM	MEANING
ADH	LSEG Real-Time Advanced Distribution Hub is the horizontally scalable service component within the LSEG Real-Time Distribution System providing high availability for publication and contribution messaging, subscription management with optional persistence, conflation and delay capabilities.
ADS	LSEG Real-Time Advanced Distribution Server is the horizontally scalable distribution component within the LSEG Real-Time Distribution System providing highly available services for tailored streaming and snapshot data, publication and contribution messaging with optional persistence, conflation and delay capabilities.
API	Application Programming Interface
ASCII	American Standard Code for Information Interchange
Enterprise Message API (EMA)	The Enterprise Message API is an ease of use, open source, Open Message Model API. EMA is designed to provide clients rapid development of applications, minimizing lines of code and providing a broad range of flexibility. It provides flexible configuration with default values to simplify use and deployment. EMA is written on top of the Enterprise Transport API utilizing the Value Added Reactor and Watchlist features of ETA.
Enterprise Transport API (ETA)	Enterprise Transport API is a high performance, low latency, foundation of the LSEG Real-Time SDK. It consists of transport, buffer management, compression, fragmentation and packing over each transport and encoders and decoders that implement the Open Message Model. Applications written to this layer achieve the highest throughput, lowest latency, low memory utilization, and low CPU utilization using a binary Rssl Wire Format when publishing or consuming content to/from LSEG Real-Time Distribution Systems.
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol (Secure)

Table 1: Acronyms and Abbreviations

ACRONYM / TERM	MEANING
JWK	JSON Web Key. Defined by RFC 7517, a JWK is a JSON formatted public or private key.
JWKS	JSON Web Key Set, This is a set of JWK, placed in a JSON array.
JWT	JSON Web Token. Defined by RFC 7519, JWT allows users to create a signed claim token that can be used to validate a user.
OMM	Open Message Model
QoS	Quality of Service
RDM	Domain Model
DP	Delivery Platform: this platform is used for REST interactions. In the context of Real-Time APIs, an API gets authentication tokens and/or queries Service Discovery to get a list of Real-Time - Optimized endpoints using DP.
LSEG Real-Time Distribution System	LSEG Real-Time Distribution System is LSEG's financial market data distribution platform. It consists of the LSEG Real-Time Advanced Distribution Server and LSEG Real-Time Advanced Distribution Hub. Applications written to the LSEG Real-Time SDK can connect to this distribution system.
Reactor	The Reactor is a low-level, open-source, easy-to-use layer above the Enterprise Transport API. It offers heartbeat management, connection and item recovery, and many other features to help simplify application code for users.
RMTES	A multi-lingual text encoding standard
RSSL	Source Sink Library
RTT	Round Trip Time, this definition is used for round trip latency monitoring feature.
RWF	Rssl Wire Format, an LSEG proprietary binary format for data representation.
LDF-D	Data Feed Direct
UML	Unified Modeling Language
UTF-8	8-bit Unicode Transformation Format

Table 1: Acronyms and Abbreviations

1.4 References

- Enterprise Message API Java Edition *LSEG Domain Model Usage Guide*
- API Concepts Guide*
- Enterprise Message API Java Edition *Configuration Guide*
- Enterprise Message API Java Edition *Developers Guide*
- Enterprise Message API Java Edition *Performance Tools Guide*
- The [LSEG Developer Community](#)

1.5 Documentation Feedback

While we make every effort to ensure the documentation is accurate and up-to-date, if you notice any errors, or would like to see more details on a particular topic, you have the following options:

- Send us your comments via email at ProductDocumentation@lseg.com.
- Add your comments to the PDF using Adobe's **Comment** feature. After adding your comments, submit the entire PDF to LSEG by clicking **Send File** in the **File** menu. Use the ProductDocumentation@lseg.com address.

1.6 Document Conventions

1.6.1 Typographic

This document uses the following types of conventions:

- Java classes, methods, in-line code snippets, and types are shown in **Courier New** font.
- Parameters, filenames, tools, utilities, and directories are shown in **Bold** font.
- Document titles and variable values are shown in *italics*.
- When initially introduced, concepts are shown in ***Bold, Italics***.
- Longer code examples are shown in Courier New font against a gray background. For example:

```
AppClient client = new AppClient();

OmmConsumerConfig config = EmaFactory.createOmmConsumerConfig();

OmmConsumer consumer =
    EmaFactory.createOmmConsumer(config.operationModel(OperationModel.USER_DISPATCH)
        .host("localhost:14002").username("user"));

ReqMsg reqMsg = EmaFactory.createReqMsg();

consumer.registerClient(reqMsg.domainType(EmaRdm.MMT_MARKET_BY_PRICE).serviceName(
    "DIRECT_FEED").name("BBH.ITS"), client);
```

1.6.2 Field and Text Values

The value for individual fields in XML files are specified as `<fieldName value="field_value"/>` where:

- **fieldName** is the name of the field and cannot contain white space.
- **field_value** sets the field's value and is always included in double quotes.

NOTE: Except for examples, double quotes are omitted from the field (parameter) descriptions throughout the remainder of this document.

Though enumerations have text values (i.e., SOCKET), in the software, text values are represented as numbers (required for programmatic configuration). When introduced, enumerations are listed along with their textual values.

1.6.3 Boolean Values

When configuring a Boolean expression, you can use any number; however Enterprise Message API interprets such expressions in the following manner:

- **0** (or any other value): false
- **1**: true

2 Enterprise Message API Configuration Overview

2.1 About Message API Configuration

The initial configuration compiled into the Enterprise Message API software defines a minimal set of configuration parameters. You can modify configuration parameters using your own custom XML file (**EmaConfig.xml**), software function calls, and programmatic interfaces. For more information on these and on configuration precedence, refer to Chapter 4, Enterprise Message API Configuration Processing.

Enterprise Message API configuration data is divided into the following groups described in Chapter 3, Configuration Groups:

- **Consumer:** Consumer configuration data are the highest-level description of the application. Such settings typically select entries from the channel, logger, and dictionary groups. For details, refer to Section 3.1.
- **IProvider:** Interactive provider configuration data is the highest-level description of the application. Such settings typically select entries from the server, logger, and directory groups. For details, refer to Section 3.2.
- **NiProvider:** Non-interactive provider configuration data is the highest-level description of the application. Such settings typically select entries from the channel, logger, and directory groups. For details, refer to Section 3.3.
- **Channel:** Channel configuration data describe various channel connection alternatives and provide configuration alternatives for those connections. For details, refer to Section 3.4.
- **WarmStandby:** Warm standby channel configuration describes a warm standby group definition which is used to enable the Warm Standby feature for consumer applications. For details, refer to Section 3.5.
- **WarmStandbyServerInfo:** Warm standby server configuration describes a channel configuration data to a starting or standby server and a set of preferred service names for the service based warm standby. For details, refer to Section 3.6.
- **SessionChannel:** Session channel configuration data specify capabilities for routing market data item requests to multiple connections for Consumer and NiProvider applications. For details, refer to Section 3.7.
- **Server:** Server configuration data describe various server connection alternatives and provide configuration alternatives for those connections. For details, refer to Section 3.8.
- **Dictionary:** Dictionary configuration data set the location information for dictionary alternatives. For details, refer to Section 3.9.
- **Directory:** Directory configuration data configure source directory refresh information. For details, refer to Section 3.10.
- **GlobalConfig:** This group contains parameters that relate to the entire application and to all instances of Consumer, Interactive and Non-interactive Providers. For details, refer to Section 3.11.

The Consumer and Provider groups are top-level configuration groups. Specific consumer and provider applications select their configurations according to the name specified in the `consumerName()` or `providerName()` method. For details on these methods, refer to Section 4.4.1.

2.2 Parameter Overview

Many default behaviors are hard-coded into the Enterprise Message API library and globally enforced. However, if you need to change API behaviors or configure the API for your specific deployment, you can use the Enterprise Message API's XML configuration file (**EmaConfig.xml**) and adjust behaviors using the appropriate parameters. While the Enterprise Message API globally enforces a set of default behaviors, certain other default behaviors are dependent on the use of the XML file and its settings.

2.3 Default Behaviors

When the Enterprise Message API library needs a parameter, it behaves according to its hard-coded configuration. You can change the API behavior by providing a valid alternate value either through the use of **EmaConfig.xml**, function calls, or programmatic methods. For default values for each of the parameters, see the appropriate Configuration Group section.

3 Configuration Groups

3.1 Consumer Group

A **ConsumerGroup** contains two elements:

- A **DefaultConsumer** element, which you can use to specify a default **Consumer** component. If a default **Consumer** is not specified in the **ConsumerGroup**, the Enterprise Message API uses the first Consumer listed in the **ConsumerList**. For details on configuring a default **Consumer**, refer to Section 3.1.2.
- A **ConsumerList** element, which contains one or more **Consumer** components (each should be uniquely identified by a **<Name .../>** entry). The consumer component is the highest-level abstraction within an application and typically refers to **Channel** and/or **Dictionary** components which specify consumer capabilities.

For a generic **ConsumerGroup** XML schema, refer to Section 3.1.1.

For details on configuring a **ConsumerGroup**, refer to Section 3.1.3.

For a list of parameters you can use in configuring a **Consumer**, refer to Section 3.1.4.

Consumer has a functionality to change certain configured parameters dynamically at runtime using IOCTL method call **OmmConsumer.modifyReactorChannelIOctl()**. For details, refer to or to the Enterprise Message API Reference Manual.

3.1.1 Generic XML Schema for ConsumerGroup

The generic XML schema for **ConsumerGroup** is as follows:

```
<ConsumerGroup>
  <DefaultConsumer value="VALUE"/>
  <ConsumerList>
    <Consumer>
      <Name value="VALUE"/>
      ...
    </Consumer>
  </ConsumerList>
</ConsumerGroup>
```

3.1.2 Setting a Default Consumer

If a **DefaultConsumer** is not specified, then the Enterprise Message API uses the first **Consumer** component in the **ConsumerGroup**. However, you can specify a default consumer by including the following parameter on a unique line inside **ConsumerGroup** but outside **ConsumerList**.

```
<DefaultConsumer value="VALUE"/>
```

3.1.3 Configuring Consumers in a ConsumerGroup

To configure a **Consumer** component, add the appropriate parameters to the target consumer in the XML schema, each on a unique line (for a list of available **Consumer** parameters, refer to Section 3.1.4).

For example, if your configuration includes channel schemas, you specify the desired channel schema by adding the following parameter inside the appropriate **Consumer** section:

```
<Channel value="VALUE"/>
```

Consumer components can use different channel schemas if the configuration includes more than one.

3.1.4 Consumer Entry Parameters

Use the following parameters when configuring a **Consumer**.

PARAMETER	TYPE	DEFAULT	DESCRIPTION
CatchUnknownJsonFids	int	1	Specifies whether the RWF/JSON conversion catches unknown JSON field IDs. Possible values: 0 (false): Do not catch unknown JSON field IDs. 1 (true): Catch unknown JSON field IDs.
CatchUnknownJsonKeys	int	0	Specifies whether the RWF/JSON conversion catches unknown JSON keys. Possible values: 0 (false): Do not catch unknown JSON keys. 1 (true): Catch unknown JSON keys.
Channel	String	N/A	Specifies the channel that the Consumer component should use. This channel must match the Name parameter from the appropriate <Channel> entry in the ChannelGroup configuration. If Channel is not specified, the Enterprise Message API resorts to default channel behavior when needed. For further details on the <Channel> entry and default behaviors, refer to Section 3.4.
ChannelSet	String	N/A	Specifies a comma-separated set of channels names. Each listed channel name should have an appropriate <Channel> entry in the ChannelGroup . Channels in the set will be tried with each reconnection attempt until a successful connection is made. For further details refer to Section 3.4.8 NOTE: If both Channel and ChannelSet are configured, then the Enterprise Message API uses the parameter that is configured last in the file. For example, if <Channel> is configured after <ChannelSet> then the Enterprise Message API uses <Channel> , but if <ChannelSet> is configured after <Channel> then the Enterprise Message API uses <ChannelSet> .
CloseChannelFromConverterFailure	int	1	Specifies that the Enterprise Message API should close the channel if the Enterprise Message API fails to parse JSON messages or if the Enterprise Message API receives JSON error messages. Possible values: 0 (false): Do not close the channel. 1 (true): Close the channel.

Table 2: Consumer Group Parameters

PARAMETER	TYPE	DEFAULT	DESCRIPTION
DefaultServiceID	int	1	Specifies a default service ID for RWF/JSON conversion if both service name and ID are missing. The maximum allowable value is 65535 .
Dictionary	String	N/A	Specifies how the consumer should access its dictionaries (it must match the Name parameter from the appropriate <Dictionary> entry in the DictionaryGroup configuration). If Dictionary is not specified, the Enterprise Message API uses the channel's dictionary when needed. For further details on this default behavior, refer to Section 3.9.
DictionaryRequestTimeout	long	45,000	Specifies the amount of time (in milliseconds) the application has to download dictionaries from a provider before the OmmConsumer throws an exception. If set to 0 , the Enterprise Message API will wait for a response indefinitely. NOTE: If ChannelSet is configured: - The Enterprise Message API honors DictionaryRequestTimeout only on its first connection. - If the channel supporting the first connection goes down, the Enterprise Message API does not use DictionaryRequestTimeout on subsequent connections.
DirectoryRequestTimeout	long	45,000	Specifies the amount of time (in milliseconds) the provider has to respond with a source directory refresh message before the OmmConsumer throws an exception. If set to 0 , the Enterprise Message API will wait for a response indefinitely. NOTE: If ChannelSet is configured: - The Enterprise Message API honors DirectoryRequestTimeout only on its first connection. - If the channel supporting the first connection goes down, the Enterprise Message API does not use DirectoryRequestTimeout on subsequent connections.
DispatchTimeoutApiThread	int	0	Specifies the duration (in microseconds) for which the internal Enterprise Message API thread is inactive before going active to check whether a message was received. If set to zero, the Enterprise Message API internal thread goes active only if it gets notified about a received message.
EmaObjectManagerComplexTypeLimit	int	-1	Specifies the size of the pools that keep EMA complex type objects (ElementList , FieldList , etc.). The value of -1 means that the pools have no limit.
EmaObjectManagerDataTypeLimit	int	-1	Specifies the size of the pools that keep EMA data type objects. The value of -1 means that the pools have no limit.
EmaObjectManagerEtaObjectsLimit	int	-1	Specifies the size of the pools that keep necessary objects from the ETA layer. The value of -1 means that the pools have no limit.
EmaObjectManagerMsgTypeLimit	int	-1	Specifies the size of the pools that keep EMA message type objects (RefreshMsg , UpdateMsg , etc.). The value of -1 means that the pools have no limit.

Table 2: Consumer Group Parameters (Continued)

PARAMETER	TYPE	DEFAULT	DESCRIPTION
EmaObjectManagerSessionObjectsLimit	int	-1	Specifies the size of the pools that keep EMA objects associated with application sessions. The value of -1 means that the pools have no limit.
EnablePreferredHostOptions	Boolean	False	This is used to enable or disable the preferred host feature. This parameter may be altered at runtime using IOCtl.
EnableRtt	long	0	Specifies whether the OmmConsumer supports gathering RoundTripLatency statistics. If enabled, the Watchlist handles automatic processing of RTT requests sent by the provider. EnableRtt expresses the consumer's consent to process RTT requests. The provider may choose either to send or not to send the requests at its own discretion. Possible values: 0 (false) - Any value > 0 (true)
ItemCountHint	long	100,000	Specifies the number of items the application expects to request. The API pre-allocates objects associated with item requests based on this value. If set to 0 , the Enterprise Message API resets it to 1024 . For better performance, the application can set this to the approximate number of item requests it expects to have registered simultaneously.
JsonExpandedEnumFields	int	0	Sets the RWF/JSON conversion to expand enumerated values in field entries to their display values for JSON protocol. Possible values: 0 (false): Do not expand enumerated fields. 1 (true): Expand enumerated fields.
LoginRequestTimeout	long	45,000	Specifies the amount of time (in milliseconds) the provider has to respond with a login refresh message before the OmmConsumer throws an exception. If set to 0 , the Enterprise Message API will wait for a response indefinitely. NOTE: If ChannelSet is configured: - The Enterprise Message API honors LoginRequestTimeout only on its first connection. - If the channel supporting the first connection goes down, the Enterprise Message API does not use LoginRequestTimeout on subsequent connections.
MaxDispatchCountApiThread	long	100	Specifies the maximum number of messages the Enterprise Message API dispatches before taking a real-time break.
MaxDispatchCountUserThread	long	100	Specifies the maximum number of messages the Enterprise Message API can dispatch in a single call to the OmmConsumer::dispatch() .
MaxOutstandingPosts	long	100,000	Specifies the maximum allowable number of on-stream posts waiting for an acknowledgment before the OmmConsumer disconnects.

Table 2: Consumer Group Parameters (Continued)

PARAMETER	TYPE	DEFAULT	DESCRIPTION
MsgKeyInUpdates	int	1	Specifies whether the Enterprise Message API fills in message key values on updates using the message key provided with the request. Possible values: • 0 (false): Do not fill in the message's key values (values received from the wire are preserved). • 1 (true): Fill in the message's key values (values received from the wire are overridden).
Name	String	N/A	Specifies the name of this Consumer component. Name is required when creating a Consumer component. You can use any value for Name .
NegativeUpdateTypeFilter	long	0	Specifies a filter on <i>types</i> of updates NOT desired. The value is a bitmask, where each bit indicates a specific update type to exclude. Possible bit values are defined in the EmaRdm class in com.refinitiv.ema.rdm package. The NegativeUpdateTypeFilter value is sent to server in the LOGIN request message to request <i>exclusion</i> of specified update types. If this is specified with UpdateTypeFilter , the server will honor only one of the filters per decode order.
ObeyOpenWindow	int	1	Specifies whether the OmmConsumer obeys the OpenWindow from services advertised in a provider's Source Directory response. Possible values: • 0 (false) • 1 (true)
PHDetectionTimeInterval	UInt64	0	Specifies time interval in seconds to switch over to a preferred host or WarmStandby group. A value of 0 indicates that the detection time interval is disabled. Precedence: if PHDetectionTimeSchedule is specified, this parameter is ignored. This parameter may be altered at runtime using IOCtl. This timer will start upon channel creation, and fire at that interval until either the channel is closed or if the interval is changed based on IOCtl.
PHDetectionTimeSchedule	String	Empty string	Specifies Cron time format to switch over to a preferred host or WarmStandby group. If this member is set to empty, PHDetectionTimeInterval is used instead if specified. This parameter may be altered at runtime using IOCtl.
PHFallbackWithinWSBGroup	Boolean	False	Specifies whether to fallback within a WarmStandby group instead of moving into a preferred WSB group. When setting PHFallbackWithinWSBGroup to false, the library closes all connections in the current WarmStandby group and attempts to connect to the preferred group. This cutover expects the starting active server to be available; if it is available, the library will connect to starting active and standby servers in the preferred group. This parameter may be altered at runtime using IOCtl.
PostAckTimeout	long	15,000	Specifies the length of time (in milliseconds) a stream waits to receive an ACK for an outstanding post before forwarding a negative acknowledgment to the application. If set to 0 , the Enterprise Message API will wait for a response indefinitely.
PreferredChannelName	String	Empty string	Specifies name of channel to set as preferred host. This parameter may be altered at runtime using IOCtl.

Table 2: Consumer Group Parameters (Continued)

PARAMETER	TYPE	DEFAULT	DESCRIPTION
PreferredWSBChannelName	String	Empty string	Specifies name of WarmStandby group to set as preferred WarmStandby group. This parameter may be altered at runtime using IOCtl.
ReconnectAttemptLimit	int	-1	Specifies the maximum number of times the consumer and non-interactive provider attempt to reconnect to a channel when it fails. If set to -1 , the consumer and non-interactive provider continually attempt to reconnect.
ReconnectMaxDelay	int	5000	Sets the maximum amount of time the consumer and non-interactive provider wait (in milliseconds) before attempting to reconnect a failed channel. Refer also to the ReconnectMinDelay parameter.
ReconnectMinDelay	int	1000	Specifies the minimum amount of time the consumer and non-interactive provider wait (in milliseconds) before attempting to reconnect a failed channel. This wait time increases with each connection attempt, from ReconnectMinDelay to ReconnectMaxDelay .
ReissueTokenAttemptInterval	int	5000	Sets the delay (in milliseconds) before the OMMConsumer attempts to reissue the token. The minimum interval is 1000 milliseconds, while the default setting is 5000.
ReissueTokenAttemptLimit	int	-1	Specifies the maximum number of times the OMMConsumer attempts to reissue the token. If set to default (i.e., -1), there is no maximum limit.
RequestTimeout	long	15,000	Specifies the amount of time (in milliseconds) the OmmConsumer waits for a response to a request before sending another request. If set to 0 , the Enterprise Message API will wait for a response indefinitely.
RestProxyHostName	String	N/A	Specifies the address or host name of the proxy server to which the Enterprise Message API connects for REST requests: service discovery and auth token service.
RestProxyPort	String	N/A	Specifies the port of the proxy server to which the Enterprise Message API connects for REST requests: service discovery and auth token service.
RestRequestTimeOut	long	45000	Specifies the timeout (in milliseconds) for token service and service discovery request. If the request times out, the OMMConsumer resends the token reissue and the timeout restarts. If the request times out, the OMMConsumer does not retry. If set to 0 , there is no timeout.
SendJsonConvError	UInt64	0	Set the RWF/JSON conversion to send back to the provider conversion error in case it occurs. Possible values: 0 (false): Do not send conversion error. 1 (true): Send conversion error.
ServiceCountHint	long	513	Sets the size of directory structures for managing services. If the application specifies 0 , the Enterprise Message API resets it to 513 .

Table 2: Consumer Group Parameters (Continued)

PARAMETER	TYPE	DEFAULT	DESCRIPTION
SessionChannelSet	String	N/A	Specifies a comma-separated set of session channel names. Each listed channel name should have an appropriate <SessionChannelInfo> entry in the SessionChannelGroup. A connection to session channels in the set will be established simultaneously. NOTE: If Channel, ChannelSet and SessionChannelSet are configured, then the Enterprise Message API uses the last SessionChannelSet in the file or programmatic configuration.
SessionEnhancedItemRecovery	int	1	Specifies that the Enterprise Message API should immediately attempt to recover the item, when the current connection goes down, if the item's requested service is available on another connection in the session channel set.
TokenReissueRatio	Double	.8	Specifies the ratio with which to multiply the access token's expiration time (in seconds) to determine the length of time the OMMConsumer waits before retrieving a new access token and refreshing its connection to Real-Time - Optimized. The valid range is from 0.05 to 0.95 .
UpdateTypeFilter	long	65533	Specifies a filter on <i>types</i> of updates desired. The value is a bitmask, where each bit indicates a specific update type. Possible bit values are defined in the EmaRdm class in com.refinitiv.ema.rdm package. The UpdateTypeFilter value is sent to server in the LOGIN request message to request <i>inclusion</i> of requested update types. If this is specified with NegativeUpdateTypeFilter, the server will honor only one of the filters per decode order.
WarmStandbyChannelSet	EmaString	N/A	Specifies a comma-separated set of warm standby channels names. Each listed name should have an appropriate WarmStandbyChannel entry in WarmStandbyGroup. Warm standby channels in the set will be moved to another warm standby channel when it failed to establish a connection for the entire warm standby channel. NOTE: - If both WarmStandbyChannelSet and ChannelSet are configured, then EMA uses the parameter that is configured in the WarmStandbyChannelSet before moving to ChannelSet. - When using both WarmStandby and ChannelSet, set reconnectAttemptLimit to a value other than default (-1) to avoid retrying servers within a WarmStandbyGroup indefinitely.
XmlTraceFileName	String	EmaTrace	Sets the name of the file to which to write XML trace output if tracing is selected.
XmlTraceMaxFileSize	long	100000000	Specifies the maximum size (in bytes) for the trace file.
XmlTracePing	int	0	Sets the Enterprise Message API to trace incoming and outgoing ping (only) messages. Possible values: 0 (false): Do not trace ping messages. 1 (true): Trace ping messages.

Table 2: Consumer Group Parameters (Continued)

PARAMETER	TYPE	DEFAULT	DESCRIPTION
XmlTraceRead	int	1	Sets the Enterprise Message API to trace incoming data. Possible values: • 0 (false): Do not trace incoming data. • 1 (true): Trace incoming data
XmlTraceToFile	int	0	If XmlTraceWrite , XmlTraceRead , and/or XmlTracePing is enabled, sets whether the Enterprise Message API traces its messages to an XML file which name is set by XmlTraceFileName . Possible values: • 0 (false): Turns off tracing. • 1 (true): Turns on tracing to an XML file.
XmlTraceToMultipleFiles	int	0	Specifies whether to write the XML trace to multiple files, if XmlTraceToFile is set. Possible values: • 1 (true): the Enterprise Message API writes the XML trace to a new file if the current file size reaches the XmlTraceMaxFileSize. • 0 (false): the Enterprise Message API stops writing the XML trace if the current file reaches the XmlTraceMaxFileSize.
XmlTraceToStdout	int	0	If XmlTraceWrite , XmlTraceRead , and/or XmlTracePing is enabled, specifies whether the Enterprise Message API traces its messages in XML format to stdout . Possible values: • 0 (false): Turns off tracing. • 1 (true): Turns on tracing to stdout.
XmlTraceWrite	int	1	Sets the Enterprise Message API to trace outgoing data. Possible values: • 0 (false): Do not trace outgoing data. • 1 (true): Trace outgoing data.

Table 2: Consumer Group Parameters (Continued)

3.2 IProvider Group

An **IProviderGroup** contains two elements that represent an Interactive Provider configuration:

- A **DefaultIProvider** element, which you can use to specify a default **IProvider** component. If a default **IProvider** is not specified in the **IProviderGroup**, the Enterprise Message API uses the first interactive provider listed in the **IProviderList**. For details on configuring a default **IProvider**, refer to Section 3.2.2.
- An **IProviderList** element, which contains one or more **IProvider** components. Each component should be uniquely identified by a **<Name .../>** entry.

The interactive provider component is the highest-level abstraction within an application. It typically refers to **Server**, **Logger**, and/or **Directory** components which specify provider capabilities.

For a generic **IProviderGroup** XML schema, refer to Section 3.3.1.

For details on configuring an **IProviderGroup**, refer to Section 3.2.3.

For a list of parameters you can use in configuring an **IProvider**, refer to Section 3.2.4.

3.2.1 Generic XML Schema for Provider Group

The generic XML schema for an **IProviderGroup** is as follows:

```
<IProviderGroup>
  <DefaultIProvider value="VALUE" />
  <IProviderList>
    <IProvider>
      <Name value="VALUE" />
      ...
    </IProvider>
  </IProviderList>
</IProviderGroup>
```

3.2.2 Setting a Default IProvider

If a **DefaultIProvider** is not specified, then the Enterprise Message API uses the first **IProvider** component in the **IProviderGroup**. However, you can specify a default provider by including the following parameter on a unique line inside the **IProviderGroup** but outside the **IProviderList**.

```
<DefaultIProvider value="VALUE" />
```

3.2.3 Configuring an IProvider in an IProviderGroup

To configure an **IProvider** component, add the appropriate parameters to the target provider in the XML schema, each on a unique line. For a list of available **IProvider** parameters, refer to Section 3.2.4.

For example, if your configuration includes channel schemas, you specify the desired channel schema by adding the following parameter inside the appropriate **IProvider** section:

```
<Channel value="VALUE"/>
```

If your provider component needs more than one channel schema, you can configure each unique schema in the XML file.

3.2.4 IProvider Entry Parameters

Use the following parameters when configuring an **IProvider**.

PARAMETER	TYPE	DEFAULT	DESCRIPTION
AcceptDirMessageWithoutMinFilters	int	0	Sets the IProvider to accept incoming directory request messages without the minimum required INFO and STATE directory filters. Possible values: 0 (false): The IProvider will not accept incoming directory messages that do not contain the minimum required INFO and STATE filters. 1 (true): The IProvider will accept incoming directory messages that do not contain the minimum required INFO and STATE filters.
AcceptMessageSameKeyButDiffStream	int	0	Sets the IProvider to accept incoming request messages even though they have a message key, domain, and private stream flag that match those of an existing request which uses a different stream ID. Possible values: 0 (false): The IProvider will not accept incoming request messages that match an existing request with a different stream ID. 1 (true): The IProvider will accept incoming request messages that match an existing request with a different stream ID.
AcceptMessageThatChangesService	int	0	Sets the IProvider to accept incoming consumer request messages on existing item stream that specify a different service name than the currently requested stream's service. Possible values: 0 (false): The IProvider will not accept incoming request messages on an existing item stream that specify a different service. 1 (true): The IProvider will accept incoming request messages on an existing item stream that specify a different service.
AcceptMessageWithoutAcceptingRequests	int	0	Sets the IProvider to accept incoming request messages even though the source directory is not accepting requests.
AcceptMessageWithoutBeingLogin	int	0	Sets the IProvider to accept incoming request messages even though the IProvider has not accepted a login request.

Table 3: IProviderGroup Parameters

PARAMETER	TYPE	DEFAULT	DESCRIPTION
AcceptMessageWithoutQosInRange	int	0	Sets the IProvider to accept incoming request messages even though the requesting QoS is not in the QoS range of the source directory.
CatchUnknownJsonFids	int	1	Specifies whether the RWF/JSON conversion catches unknown JSON field IDs. Possible values: 0 (false): Do not catch unknown JSON field IDs. 1 (true): Catch unknown JSON field IDs.
CatchUnknownJsonKeys	int	0	Specifies whether the RWF/JSON conversion catches unknown JSON keys. Possible values: 0 (false): Do not catch unknown JSON keys. 1 (true): Catch unknown JSON keys.
ClientSessionCountHint	int	10	Specifies the number of client sessions the application expects to maintain. The API will pre-allocate the minimum value of ClientSessionCountHint and ClientSessionPoolLimit . Formula for the number of objects that the API pre-allocates: $\text{MinimumOf}(\text{ClientSessionPoolLimit}, \text{ClientSessionCountHint})$
ClientSessionPoolLimit	int	-1	Specifies the maximum number of client session objects the pool will store at a time. The default value of -1 means that no limit is set. Formula for the number of objects that the API pre-allocates: $\text{MinimumOf}(\text{ClientSessionPoolLimit}, \text{ClientSessionCountHint})$
CloseChannelFromConverterFailure	int	1	Specifies that the Enterprise Message API should close the channel if it fails to parse JSON messages or if it receives JSON error messages. Possible values: 0 (false): Do not close the channel. 1 (true): Close the channel.
DefaultServiceID	int	1	Specifies a default service ID for RWF/JSON conversion if both service name and ID are missing. The maximum allowable value is 65535 .
Directory	String	N/A	Specifies source directory refresh information that the IProvider sends after establishing a connection. This must match the Name parameter from the appropriate <Directory> entry in the DirectoryGroup configuration. If Directory is not specified, the Enterprise Message API uses a hard coded configuration. For details on the <Directory> entry and default settings, refer to Section 3.10.
DispatchTimeoutApiThread	int	0	Specifies the duration (in microseconds) for which the internal Enterprise Message API thread is inactive before going active to check whether a message was received. If set to zero, the thread goes active only if notified about a received message.
EmaObjectManagerComplexTypeLimit	int	-1	Specifies the size of the pools that keep EMA complex type objects (ElementList , FieldList , etc.). The value of -1 means that the pools have no limit.
EmaObjectManagerDataTypeLimit	int	-1	Specifies the size of the pools that keep EMA data type objects. The value of -1 means that the pools have no limit.

Table 3: IProviderGroup Parameters (Continued)

PARAMETER	TYPE	DEFAULT	DESCRIPTION
EmaObjectManagerEtaObjectsLimit	int	-1	Specifies the size of the pools that keep necessary objects from the ETA layer. The value of -1 means that the pools have no limit.
EmaObjectManagerMsgTypeLimit	int	-1	Specifies the size of the pools that keep EMA message type objects (RefreshMsg , UpdateMsg , etc.). The value of -1 means that the pools have no limit.
EmaObjectManagerSessionObjectsLimit	int	-1	Specifies the size of the pools that keep EMA objects associated with application sessions. The value of -1 means that the pools have no limit.
EnforceAckIDValidation	int	0	Specifies whether IProvider has to validate the AckId attribute when an AckMsg calls OmmIProvider::submit() . If validation is turned on, then AckId must be equal to the PostId of PostMsg received by the IProvider . Possible values: 1 (true): Validate the AckId. 0 (false): Do not validate the AckId.
EnumTypeFragmentSize	int	21000	Sets the maximum fragmentation size (in bytes) of enumerated types dictionary multi-part refresh messages.
FieldDictionaryFragmentSize	int	8192	Sets the maximum fragmentation size (in bytes) of field dictionary multi-part refresh messages.
ItemCountHint	int	100,000	Specifies the number of items the application expects to maintain per client session. The API pre-allocates objects associated with item requests based on this value. If set to 0 , Enterprise Message API resets it to 1024 . For better performance, the application can set this to the approximate number of items it maintains per client session. See ItemInfoPoolLimit definition for more information
ItemInfoPoolLimit	int	-1	Specifies the number of ItemInfo objects associated with item requests that the application expects to maintain. The API will grow objects up to this ItemInfoPoolLimit . Formula for the number of objects that the API pre-allocates: MinimumOf (ItemInfoPoolLimit, (ItemCountHint x ClientSessionCountHint)) .
JsonExpandedEnumFields	int	0	Sets the RWF/JSON conversion to expand enumerated values in field entries to their display values for JSON protocol. Possible values: 0 (false): Do not expand enumerated fields. 1 (true): Expand enumerated fields.
MaxDispatchCountApiThread	int	100	Specifies the maximum number of messages the Enterprise Message API dispatches before taking a real-time break.
MaxDispatchCountUserThread	int	100	Specifies the maximum number of messages the Enterprise Message API can dispatch in a single call to the OmmIProvider::dispatch() .
Name	String	N/A	Specifies the name of this IProvider component. Required when creating an IProvider component. You can use any value for Name .

Table 3: IProviderGroup Parameters (Continued)

PARAMETER	TYPE	DEFAULT	DESCRIPTION
RefreshFirstRequired	int	1	Specifies whether the Enterprise Message API requires the application to send a refresh message prior to sending update messages. Possible values: • 1 (true): The IProvider does not require that a refresh message is sent prior to update messages. • 0 (false): The IProvider requires that a refresh message is sent prior to update messages.
RequestTimeout	int	15000	Specifies the length of time (in milliseconds) the OmmIProvider waits for a response to a request before sending another request. The DICTIONARY domain will not send another request. If set to 0 , the Message API waits for a response indefinitely.
SendJsonConvError	int	0	Set the RWF/JSON conversion to send back to the consumer conversion error in case it occurs. Possible values: • 0 (false): Do not send conversion error. • 1 (true): Send conversion error.
Server	String	N/A	Specifies the channel that the IProvider component should use. This channel must match the Name parameter from the appropriate <Server> entry in the ServerGroup configuration. If Server is not specified, the Enterprise Message API resorts to default channel behavior when needed. For details on the <Server> entry and default behaviors, refer to Section 3.8.
ServiceCountHint	int	513	Sets the size of directory structures for managing services. If the application specifies 0 , the Enterprise Message API resets it to 513 .
XmlTraceFileName	String	EmaTrace	Sets the name of the file to which to write XML trace output if tracing is selected.
XmlTraceMaxFileSize	int	100000000	Specifies the maximum size (in bytes) for the trace file.
XmlTracePing	int	0	Sets the Enterprise Message API to trace incoming and outgoing ping (only) messages. Possible values: • 0 (false): Do not trace ping messages. • 1 (true): Trace ping messages.
XmlTraceRead	int	1	Sets the Enterprise Message API to trace incoming data. Possible values: • 0 (false): Do not trace incoming data. • 1 (true): Trace incoming data
XmlTraceToFile	int	0	If XmlTraceWrite , XmlTraceRead , and/or XmlTracePing is enabled, sets whether the Enterprise Message API traces its messages to an XML file whose name is set by XmlTraceFileName . Possible values: • 0 (false): Turns off tracing. • 1 (true): Turns on tracing to an XML file.

Table 3: IProviderGroup Parameters (Continued)

PARAMETER	TYPE	DEFAULT	DESCRIPTION
XmlTraceToMultipleFiles	int	0	Specifies whether to write the XML trace to multiple files, if XmlTraceToFile is set. Possible values: 1 (true): the Enterprise Message API writes the XML trace to a new file if the current file size reaches the XmlTraceMaxFileSize. 0 (false): the Enterprise Message API stops writing the XML trace if the current file reaches the XmlTraceMaxFileSize.
XmlTraceToStdout	int	0	If XmlTraceWrite , XmlTraceRead , and/or XmlTracePing is enabled, specifies whether the Enterprise Message API traces its messages in XML format to stdout. Possible values: 0 (false): Turns off tracing. 1 (true): Turns on tracing to stdout.
XmlTraceWrite	int	1	Sets the Enterprise Message API to trace outgoing data. Possible values: 0 (false): Do not trace outgoing data. 1 (true): Trace outgoing data.

Table 3: IProviderGroup Parameters (Continued)

3.3 NiProvider Group

An **NiProviderGroup** contains two elements that represent a Non-interactive Provider configuration:

- A **DefaultNiProvider** element, which you can use to specify a default **NiProvider** component. If a default **NiProvider** is not specified in the **NiProviderGroup**, the Enterprise Message API uses the first non-interactive provider listed in the **NiProviderList**. For details on configuring a default **NiProvider**, refer to Section 3.3.2.
- An **NiProviderList** element, which contains one or more **NiProvider** components. Each component should be uniquely identified by a **<Name .../>** entry).

The non-interactive provider component is the highest-level abstraction within an application. It typically refers to **Channel**, **Logger**, and/or **Directory** components which specify provider capabilities.

For a generic **NiProviderGroup** XML schema, refer to Section 3.3.1.

For details on configuring an **NiProviderGroup**, refer to Section 3.3.3.

For a list of parameters you can use in configuring an **NiProvider**, refer to Section 3.3.4.

3.3.1 Generic XML Schema for NiProvider Group

The generic XML schema for an **NiProviderGroup** is as follows:

```
<NiProviderGroup>
  <DefaultNiProvider value="VALUE" />
  <NiProviderList>
    <NiProvider>
      <Name value="VALUE" />
      ...
    </NiProvider>
  </NiProviderList>
</NiProviderGroup>
```

3.3.2 Setting a Default NiProvider

If a **DefaultNiProvider** is not specified, then the Enterprise Message API uses the first **Provider** component in the **NiProviderGroup**. However, you can specify a default provider by including the following parameter on a unique line inside the **NiProviderGroup** but outside the **NiProviderList**.

```
<DefaultNiProvider value="VALUE" />
```

3.3.3 Configuring an NiProvider in an NiProviderGroup

To configure an **NiProvider** component, add the appropriate parameters to the target provider in the XML schema, each on a unique line. For a list of available **NiProvider** parameters, refer to Section 3.3.4.

For example, if your configuration includes channel schemas, you specify the desired channel schema by adding the following parameter inside the appropriate **NiProvider** section:

```
<Channel value="VALUE"/>
```

If your non-interactive provider component needs more than one channel schema, you can configure each unique schema in the XML file.

3.3.4 NiProvider Entry Parameters

Use the following parameters when configuring an **NiProvider**.

PARAMETER	TYPE	DEFAULT	DESCRIPTION
CatchUnknownJsonFids	int	1	Specifies whether the RWF/JSON conversion catches unknown JSON field IDs. Possible values: 0 (false): Do not catch unknown JSON field IDs. 1 (true): Catch unknown JSON field IDs.
CatchUnknownJsonKeys	int	0	Specifies whether the RWF/JSON conversion catches unknown JSON keys. Possible values: 0 (false): Do not catch unknown JSON keys. 1 (true): Catch unknown JSON keys.
Channel	String	N/A	Specifies the channel that the NiProvider component should use. This channel must match the Name parameter from the appropriate <Channel> entry in the ChannelGroup configuration. If Channel is not specified, the Enterprise Message API resorts to default channel behavior when needed. For further details on the <Channel> entry and default behaviors, refer to Section 3.4.
ChannelSet	String	N/A	Specifies a comma-separated set of channel names. Each channel name must have a corresponding <Channel> entry in the ChannelGroup . In the event of a reconnection, Channels in the set are tried until a successful connection is made. For further details, refer to Section 3.4.8. NOTE: If both Channel and ChannelSet are configured, the Enterprise Message API uses the parameter configured last (linearly) in the file. For example: - If <Channel> is configured after <ChannelSet>, the Enterprise Message API uses <Channel>. - If <ChannelSet> is configured after <Channel>, the Enterprise Message API uses <ChannelSet>.

Table 4: NiProviderGroup Parameters

PARAMETER	TYPE	DEFAULT	DESCRIPTION
CloseChannelFromConverterFailure	int	1	Specifies that the Enterprise Message API should close the channel if it fails to parse JSON messages or if it receives JSON error messages. Possible values: 0 (false): Do not close the channel. 1 (true): Close the channel.
DefaultServiceID	int	1	Specifies a default service ID for RWF/JSON conversion if both service name and ID are missing. The maximum allowable value is 65535 .
Directory	String	N/A	Specifies source directory refresh information that the NiProvider sends after establishing a connection. This must match the Name parameter from the appropriate <Directory> entry in the DirectoryGroup configuration. If Directory is not specified, the Enterprise Message API uses a hard coded configuration. For further details on the <Directory> entry and default settings, refer to Section 3.10.
DispatchTimeoutApiThread	int	0	Specifies the duration (in microseconds) for which the internal Enterprise Message API thread is inactive before going active to check whether a message was received. If set to zero, the thread goes active only if notified about a received message.
EmaObjectManagerComplexTypeLimit	int	-1	Specifies the size of the pools that keep EMA complex type objects (ElementList , FieldList , etc.). The value of -1 means that the pools have no limit.
EmaObjectManagerDataTypeLimit	int	-1	Specifies the size of the pools that keep EMA data type objects. The value of -1 means that the pools have no limit.
EmaObjectManagerEtaObjectsLimit	int	-1	Specifies the size of the pools that keep necessary objects from the ETA layer. The value of -1 means that the pools have no limit.
EmaObjectManagerMsgTypeLimit	int	-1	Specifies the size of the pools that keep EMA message type objects (RefreshMsg , UpdateMsg , etc.). The value of -1 means that the pools have no limit.
EmaObjectManagerSessionObjectsLimit	int	-1	Specifies the size of the pools that keep EMA objects associated with application sessions. The value of -1 means that the pools have no limit.
ItemCountHint	int	100,000	Specifies the number of items the application expects to maintain. The API pre-allocates objects associated with the maintained items based on this value. If set to 0 , the Enterprise Message API resets it to 1024 . For better performance, the application can set this to the approximate number of items it maintains.
JsonExpandedEnumFields	int	0	Sets the RWF/JSON conversion to expand enumerated values in field entries to their display values for JSON protocol. Possible values: 0 (false): Do not expand enumerated fields. 1 (true): Expand enumerated fields.

Table 4: NiProviderGroup Parameters (Continued)

PARAMETER	TYPE	DEFAULT	DESCRIPTION
LoginRequestTimeout	int	45,000	Specifies the amount of time (in milliseconds) the consuming component has to respond with a login refresh message before the OmmNiProvider throws an exception. If set to 0, the Enterprise Message API will wait for a response indefinitely. NOTE: When ChannelSet is configured, the Enterprise Message API honors LoginRequestTimeout only on its first connection. If the channel supporting the first connection goes down, the Enterprise Message API does not use LoginRequestTimeout on subsequent connections.
MaxDispatchCountApiThread	int	100	Specifies the maximum number of messages the Enterprise Message API dispatches before taking a real-time break.
MaxDispatchCountUserThread	int	100	Specifies the maximum number of messages the Enterprise Message API can dispatch in a single call to the OmmNiProvider::dispatch() .
MergeSourceDirectoryStreams	int	1	Specifies if the Enterprise Message API merges all source directory streams (configured and user-submitted) into one stream. Possible values: • 1 (true) • 0 (false)
Name	String	N/A	Specifies the name of this NiProvider component. Name is required when creating an NiProvider component. You can use any value for Name .
ReconnectAttemptLimit	int	-1	Specifies the maximum number of times the non-interactive provider will attempt to reconnect to a channel when it fails. If set to -1 , the non-interactive provider continually attempts to reconnect.
ReconnectMaxDelay	int	5000	Sets the maximum amount of time the non-interactive provider will wait (in milliseconds) before attempting to reconnect a failed channel. Refer also to the ReconnectMinDelay parameter.
ReconnectMinDelay	int	1000	Specifies the minimum amount of time the non-interactive provider will wait (in milliseconds) before attempting to reconnect a failed channel. This wait time increases with each connection attempt, from ReconnectMinDelay to ReconnectMaxDelay .
RecoverUserSubmitSourceDirectory	int	1	Specifies whether the Enterprise Message API recovers user-submitted source directories when recovering from a disconnect. Possible values: • 1 (true): The API will recover user-submitted source directories automatically on recovery from a disconnect. • 0 (false): The API will not recover user-submitted source directories.

Table 4: NiProviderGroup Parameters (Continued)

PARAMETER	TYPE	DEFAULT	DESCRIPTION
RefreshFirstRequired	int	1	Specifies whether the Enterprise Message API requires the application to send a refresh message prior to sending update messages. Possible values: 1 (true): The NiProvider does not require that a refresh message is sent prior to update messages. 0 (false): The NiProvider requires that a refresh message is sent prior to update messages.
RemoveItemsOnDisconnect	int	1	Specifies whether the Enterprise Message API removes items from its internal hash table whenever it disconnects from the LSEG Real-Time Advanced Distribution Hub. Possible values: 1 (true) 0 (false)
RequestTimeout	int	15000	Specifies the length of time (in milliseconds) the OmniNiProvider waits for a response to a request before sending another request. The DICTIONARY domain will not send another request. If set to 0 , the Message API waits for a response indefinitely.
SendJsonConvError	int	0	Set the RWF/JSON conversion to send back to the consumer conversion error when it occurs. Possible values are: 0 (false): Do not send conversion error. 1 (true): Send conversion error.
ServiceCountHint	int	513	Sets the size of directory structures for managing services. If the application specifies 0 , the Enterprise Message API resets it to 513 .
SessionChannelSet	String	N/A	Specifies a comma-separated set of session channel names. Each listed channel name should have an appropriate <SessionChannelInfo> entry in the SessionChannelGroup . A connection to session channels in the set will be established simultaneously. NOTE: If Channel , ChannelSet , and SessionChannelSet are configured, the Enterprise Message API uses the last SessionChannelSet in the file or programmatic configuration. NOTE: Specifying a SessionChannelSet will set RecoverUserSubmitSourceDirectory to true for the NiProvider instance.
XmlTraceFileName	String	EmaTrace	Sets the name of the file to which to write XML trace output if tracing is selected.
XmlTraceMaxFileSize	int	100000000	Specifies the maximum size (in bytes) for the trace file.
XmlTracePing	int	0	Sets the Enterprise Message API to trace incoming and outgoing ping (only) messages. Possible values: 0 (false): Do not trace ping messages. 1 (true): Trace ping messages.

Table 4: NiProviderGroup Parameters (Continued)

PARAMETER	TYPE	DEFAULT	DESCRIPTION
XmlTraceRead	int	1	Sets the Enterprise Message API to trace incoming data. Possible values: • 0 (false): Do not trace incoming data. • 1 (true): Trace incoming data
XmlTraceToFile	int	0	If XmlTraceWrite , XmlTraceRead , and/or XmlTracePing is enabled, sets whether the Enterprise Message API traces its messages to an XML file whose name is set by XmlTraceFileName . Possible values: • 0 (false): Turns off tracing. • 1 (true): Turns on tracing to an XML file.
XmlTraceToMultipleFiles	int	0	Specifies whether to write the XML trace to multiple files, if XmlTraceToFile is set. Possible values: • 1 (true): the Enterprise Message API writes the XML trace to a new file if the current file size reaches the XmlTraceMaxFileSize. • 0 (false): the Enterprise Message API stops writing the XML trace if the current file reaches the XmlTraceMaxFileSize.
XmlTraceToStdout	int	0	If XmlTraceWrite , XmlTraceRead , and/or XmlTracePing is enabled, specifies whether the Enterprise Message API traces its messages in XML format to stdout. Possible values: • 0 (false): Turns off tracing. • 1 (true): Turns on tracing to stdout.
XmlTraceWrite	int	1	Sets the Enterprise Message API to trace outgoing data. Possible values: • 0 (false): Do not trace outgoing data. • 1 (true): Trace outgoing data.

Table 4: NiProviderGroup Parameters (Continued)

3.4 Channel Group

ChannelGroup is used only with the **Consumer** and **NiProvider**.

The **ChannelGroup** contains a **ChannelList**, which contains one or more **Channel** entries (each uniquely identified by a **<Name .../>** entry). Each channel includes a set of connection parameters for a specific connection or connection type.

There is no default channel. If an Enterprise Message API application needs a specific channel, you must specify this in the appropriate **Consumer** or **NiProvider** section.

- For details on the parameters you can use to configure the **Consumer** component, refer to Section 3.1.4.
- For details on the parameters you can use to configure the **NiProvider** component, refer to Section 3.3.4.
- For a generic **ChannelGroup** XML schema, refer to Section 3.4.1.
- For a list of universal parameters you can use in configuring any type of **Channel** regardless of the channel type, refer to Section 3.4.2.
- For a list of parameters you can use only when configuring a **Channel** whose channel type is **RSSL_SOCKET**, refer to Section 3.4.4.
- For a list of parameters you can use only when configuring a **Channel** whose channel type is **RSSL_ENCRYPTED**, refer to Section 3.4.5.
- For a list of parameters you can use only when configuring a **Channel** whose channel type is **RSSL_HTTP**, refer to Section 3.4.5.

3.4.1 Generic XML Schema for ChannelGroup

The top-level XML schema for the **ChannelGroup** is as follows:

```
<ChannelGroup>
  <ChannelList>
    <Channel>
      <Name value="VALUE"/>
      ...
    </Channel>
  </ChannelList>
</ChannelGroup>
```

3.4.2 Universal Channel Entry Parameters

You can use the following parameters in any **<Channel>** entry, regardless of the **ChannelType**.

For additional information on how to set the **Channel** connection type using the **ChannelType** and **EncryptedProtocolType** parameters, refer to Section 3.4.3.

PARAMETER NAME	TYPE	DEFAULT	NOTES
ChannelType	String	RSSL_SOCKET	Specifies the type of channel or connection used to connect to the server. Calling the host function can change this field. For details on this event, refer to Section 4.4.2. Use strings with Enterprise Message API's programmatic configuration (for further details, refer to Section 4.6). Available values include: • RSSL_SOCKET • RSSL_ENCRYPTED • RSSL_HTTP • RSSL_WEBSOCKET
ConnectionPingTimeout	int	30000	Specifies the duration (in milliseconds) after which the Enterprise Message API terminates the connection if it does not receive communication or pings from the server.
DirectWrite	int	0	Specifies whether to set the direct socket write flag when sending data on a channel. When the flag is set, every package is sent on the wire immediately on the submit call. If direct write is not set, the package might be placed into an internal queue which is later flushed onto the wire. Possible values are: • 0: Send data without the direct socket write flag. • 1: Send data with the direct socket write flag.
EnableSessionManagement	int	0	Specifies whether the channel manages the authentication token on behalf of the user. If set to 1, the channel obtains the authentication token and refreshes it as needed on behalf of the user. The default setting is 0. You can use this parameter only in with Enterprise Message API consumers. When EnableSessionManagement is set and used with implicit Service Discovery, the application must configure ChannelType to be RSSL_ENCRYPTED because endpoints obtained by querying RDP Service Discovery are encrypted endpoints.
GuaranteedOutputBuffers	int	100	Specifies the number of guaranteed buffers (allocated at initialization time) available for use by each RsslChannel when writing data. Each buffer is created to contain maxFragmentSize bytes. For details on RsslChannel and maxFragmentSize, refer to the <i>Transport API Java Edition Developers Guide</i>.
HighWaterMark	int	6144	Specifies the upper buffer-usage threshold for the channel. Must be set explicitly in either file or programmatic configuration.

Table 5: Universal <Channel> Parameters

PARAMETER NAME	TYPE	DEFAULT	NOTES
InitializationTimeout	int	5 (10 when used with RSSL_ENCRYPTED ChannelType)	Specifies the time (in seconds) to wait for the successful initialization of a channel.
InterfaceName	String	""	Specifies a character representation of the IP address or hostname of the local network interface over which the Enterprise Message API sends and receives content. InterfaceName is for use in systems that have multiple network interface cards. If unspecified, the default network interface is used.
Location	String	us-east-1	Used only when host and port are unspecified, Location specifies the cloud location of the service provider endpoint to which the RTSDK API establishes a connection. If Location is not specified, the default setting is us-east-1 . In any particular cloud location, the Enterprise Message API connects to the endpoint that provides two available zones for the location (e.g., [us-east-1a , us-east-1b]). You can use Location only on an RSSL_ENCRYPTED ChannelType.
Name	String		Specifies the Channel 's name.
NumInputBuffers	int	100	Specifies the number of buffers used to read data. Buffers are sized according to maxFragmentSize . For details on RsslChannel and maxFragmentSize , refer to the <i>Transport API Java Edition Developers Guide</i> .
ServiceDiscoveryRetryCount	int	3	Specifies the number of times the RTSDK API attempts to reconnect a channel before forcing the API to retry service discovery. Used only when: - Host and port are unspecified. Refer to Section 3.4.4. EnableSessionManagement is set to 1. For details on service discovery, refer to the <i>Enterprise Message API Java Edition Developers Guide</i> .
			NOTE: You can use this parameter only with Enterprise Message API consumers. API will not retry to get an endpoint from the service discovery when the value is 0 .
SysRecvBufSize	int	65535	Specifies the size (in bytes) of the system's receive buffer for this channel. For exact, effective values, refer to your operating system documentation.
SysSendBufSize	int	65535	Specifies the size (in bytes) of the system's send buffer for this channel. For exact, effective values, refer to your operating system documentation.

Table 5: Universal <Channel> Parameters (Continued)

3.4.3 EMA Channel Connection Types

Following are sample snippets from the configuration file that show how to set up the Channel connection type:

```
<EncryptedProtocolType value="EncryptedProtocolType::RSSL_WEBSOCKET"/>
<ChannelType value="ChannelType::RSSL_ENCRYPTED"/>
```

The following table summarizes possible Channel connection types and parameter values that you can use to set them.

CHANNEL CONNECTION TYPE	CHANNELTYPE	ENCRYPTEDPROTOCOLTYPE	SUBPROTOCOL LIST POSSIBILITIES
Unencrypted Websocket	RSSL_WEBSOCKET	Not used	The following possibilities apply: - If the WsProtocols parameter is empty or mismatched with server support, websocket connections are denied. - Default: WsProtocols value="tr_json2, rssl.rwf, rssl.json.v2"
Encrypted Websocket	RSSL_ENCRYPTED	RSSL_WEBSOCKET	
Unencrypted Socket	RSSL_SOCKET	Not used	Not used; RWF is implied.
Encrypted Socket	RSSL_ENCRYPTED	RSSL_SOCKET	

Table 6: Channel Settings for Socket and Websocket Connection Types

3.4.4 Parameters for Use with Channel Type: RSSL_SOCKET

In addition to the universal parameters listed in Section 3.4.2, you can use the following parameters to configure a channel whose type is **RSSL_SOCKET**.

PARAMETER NAME	TYPE	DEFAULT	NOTES
CompressionThreshold	long	30	Sets the message size threshold (in bytes, the allowed value is 30-Integer.MAX_VALUE), above which all messages are compressed (thus individual messages might not be compressed). Different compression types have different behaviors and compression efficiency can vary depending on message size.
CompressionType	String	None	Specifies the Enterprise Message API's preferred type of compression. Compression is negotiated between the client and server: if the server supports the preferred compression type, the server will compress data at that level. Use strings with Enterprise Message API's programmatic configuration (for further details, refer to Section 4.6). Available values include: None ZLib LZ4
			NOTE: A server can be configured to force a particular compression type, regardless of client settings.

Table 7: Parameters for Channel Type: RSSL_SOCKET

PARAMETER NAME	TYPE	DEFAULT	NOTES
DirectWrite		0	Specifies whether to set the direct socket write flag when sending data on a channel. When the flag is set, every package is sent on the wire immediately on the submit call. If direct write is not set, the package might be placed into an internal queue which is later flushed onto the wire. Possible values are: • 0: Send data without the direct socket write flag. • 1: Send data with the direct socket write flag.
Host	String	localhost	Specifies the host name of the server to which the Enterprise Message API connects. The parameter value can be a remote host name or IP address.
Port	String	14002	Specifies the port on the remote server to which the Enterprise Message API connects.
ProxyHost	String	""	Specifies the host name of the proxy to which the Enterprise Message API connects. The parameter value can be a host name or an IP address. Any value provided by a function call overrides the setting in configuration file. NOTE: This parameter can be used to invoke REST requests such as service discovery and obtain an authentication token. They have lower precedence than the proxy settings (RestProxyHostName) specified in the Consumer group. Refer to Section 3.1.4.
ProxyPort	String	""	Specifies the port on the proxy to which the Enterprise Message API connects. Any value provided by a function call overrides the setting in configuration file. NOTE: This parameter can be used to invoke REST requests such as service discovery and obtain an authentication token. They have lower precedence than the proxy settings (RestProxyPort) specified in the Consumer group. Refer to Section 3.1.4.
TcpNodelay	int	1	Specifies whether to use Nagle's algorithm when sending data. Available values are: • 0: Send data using Nagle's algorithm. • 1: Send data without delay.

Table 7: Parameters for Channel Type: RSSL_SOCKET (Continued)

3.4.5 Parameters for Use with Channel Types: RSSL_HTTP

In addition to the universal parameters listed in Section 3.4.2, you can use the following parameters to configure a channel whose type is **RSSL_HTTP**.

PARAMETER NAME	TYPE	DEFAULT	NOTES
CompressionThreshold	int	30	Sets the message size threshold (in bytes, the allowed value is 30-UInt32 MAX), above which all messages are compressed (thus individual messages might not be compressed). Different compression types have different behaviors and compression efficiency can vary depending on message size.
CompressionType	String	None	Specifies the Enterprise Message API's preferred type of compression. Compression is negotiated between the client and server: if the server supports the preferred compression type, the server will compress data at that level. Use strings with Enterprise Message API's programmatic configuration (for further details, refer to Section 4.6). Available values include: • None • ZLib • LZ4 NOTE: A server can be configured to force a particular compression type, regardless of client settings.
Host	String	localhost	Specifies the host name of the server to which the Enterprise Message API connects. The parameter value can be a remote host name or IP address.
ObjectName	String	""	Specifies the object name to pass along with the underlying URL in HTTP and HTTPS connection messages.
Port	String	14002	Specifies the port on the remote server to which the Enterprise Message API connects.
ProxyHost	String	""	Specifies the host name of the proxy to which the Enterprise Message API connects. The parameter value can be a host name or an IP address. Any value provided by a function call overrides the setting in configuration file. NOTE: This parameter can be used to invoke REST requests such as service discovery and obtain an authentication token. They have lower precedence than the proxy settings (RestProxyHostName) specified in the Consumer group. Refer to Section 3.1.4.
ProxyPort	String	""	Specifies the port on the proxy to which the Enterprise Message API connects. Any value provided by a function call overrides the setting in configuration file. NOTE: This parameter can be used to invoke REST requests such as service discovery and obtain an authentication token. They have lower precedence than the proxy settings (RestProxyPort) specified in the Consumer group. Refer to Section 3.1.4.
TcpNodelay	int	1	Specifies whether to use Nagle's algorithm when sending data. Available values are: • 0: Send data using Nagle's algorithm. • 1: Send data without delay.

Table 8: Parameters for Channel Type: RSSL_HTTP

3.4.6 Parameters for Use with Channel Types: **RSSL_WEBSOCKET**

In addition to the universal parameters listed in Section 3.4.2, you can use the following parameters to configure a channel whose type is **RSSL_WEBSOCKET**. A **RSSL_WEBSOCKET** channel does not support the LZ4 compression type.

PARAMETER NAME	TYPE	DEFAULT	NOTES
WsMaxMsgSize	int	61440	Specifies the maximum size of messages that the WebSocket transport can send or read.
WsProtocols	String	tr_json2, rssl.rwf, rssl.json.v2	Specifies a list of supported/preferred protocols in order of preference from highest to lowest.

Table 9: Parameters for Channel Types: **RSSL_WEBSOCKET**

3.4.7 Parameters for Use with Channel Types: **RSSL_ENCRYPTED**

In addition to the universal parameters listed in Section 3.4.2, and the parameters listed in the section specific to the protocol type you use (i.e., Section 3.4.4 for socket connections or Section 3.4.5 for HTTP connections), use the following parameters to configure a channel whose type is **RSSL_ENCRYPTED**.

PARAMETER NAME	TYPE	DEFAULT	NOTES
EncryptedProtocolType	String	RSSL_SOCKET	Specifies the type of protocol used for this encrypted connection. RSSL_SOCKET (0) RSSL_HTTP (2) RSSL_WEBSOCKET (7)

Table 10: Parameters for Channel Types: **RSSL_ENCRYPTED**

3.4.8 Example XML Schema for Configuring ChannelSet

The following is an example XML schema for use in configuring a **ChannelSet**:

```
<ConsumerGroup>
  <ConsumerList>
    <Consumer>
      ...
      <!-- ChannelSet specifies an ordered list of Channels to which OmmConsumer will attempt -->
      <!-- to connect, one at a time, if the previous one fails to connect -->
      <ChannelSet value="VALUE1,  VALUE2, ..."/>
      ...
    </Consumer>
  </ConsumerList>
</ConsumerGroup>
```

3.4.9 Example ChannelSet XML Configuration

The following XML example illustrates a specific ChannelSet configuration using the XML schema introduced in Section 3.4.8:

```
<ConsumerGroup>
  <ConsumerList>
    <Consumer>
      <Name value="Consumer_1"/>
      <!-- ChannelSet specifies an ordered list of Channels to which OmmConsumer will attempt -->
      <!-- to connect, one at a time, if the previous one fails to connect -->
      <ChannelSet value="Channel_1, Channel_2"/>
      <ReconnectAttemptLimit value="10"/>
      <XmlTraceToStdout value="1"/>
    </Consumer>
  </ConsumerList>
</ConsumerGroup>
<ChannelGroup>
  <ChannelList>
    <Channel>
      <Name value="Channel_1"/>
      <ChannelType value="ChannelType::RSSL_SOCKET"/>
      <Host value="localhost"/>
      <Port value="14002"/>
    </Channel>
    <Channel>
      <Name value="Channel_2"/>
      <ChannelType value="ChannelType::RSSL_SOCKET"/>
      <Host value=" localhost "/>
      <Port value="14008"/>
    </Channel>
  </ChannelList>
</ChannelGroup>
```

3.4.10 Example Programmatic Configuration for ChannelSet

The following XML example illustrates a programmatic ChannelSet configuration in Java:

```
Map innerMap = EmaFactory.createMap();
Map configMap = EmaFactory.createMap();
ElementList elementList = EmaFactory.createElementList();
ElementList innerElementList = EmaFactory.createElementList();

elementList.add(EmaFactory.createElementEntry().ascii("DefaultConsumer", "Consumer_1"));
innerElementList.add(EmaFactory.createElementEntry().ascii("ChannelSet", "Channel_1, Channel_2"));
innerElementList.add(EmaFactory.createElementEntry().ascii("Dictionary", "Dictionary_1"));
innerMap.add(EmaFactory.createMapEntry().keyAscii( "Consumer_1", MapEntry.MapAction.ADD,
    innerElementList));
innerElementList.clear();

elementList.add(EmaFactory.createElementEntry().map( "ConsumerList", innerMap ));
innerMap.clear();
configMap.add(EmaFactory.createMapEntry().keyAscii( "ConsumerGroup", MapEntry.MapAction.ADD,
    elementList ));
elementList.clear();

innerElementList.add(EmaFactory.createElementEntry().ascii("ChannelType",
    "ChannelType::RSSL_SOCKET"));
innerElementList.add(EmaFactory.createElementEntry().ascii("Host", "localhost"));
innerElementList.add(EmaFactory.createElementEntry().ascii("Port", "14002"));
innerMap.add(EmaFactory.createMapEntry().keyAscii( "Channel_1", MapEntry.MapAction.ADD,
    innerElementList));
innerElementList.clear();

innerElementList.add(EmaFactory.createElementEntry().ascii("ChannelType",
    "ChannelType::RSSL_SOCKET"));
innerElementList.add(EmaFactory.createElementEntry().ascii("Host", "121.1.1.100"));
innerElementList.add(EmaFactory.createElementEntry().ascii("Port", "14008"));
innerMap.add(EmaFactory.createMapEntry().keyAscii( "Channel_2", MapEntry.MapAction.ADD,
    innerElementList));
innerElementList.clear();

elementList.add(EmaFactory.createElementEntry().map( "ChannelList", innerMap ));
innerMap.clear();

configMap.add(EmaFactory.createMapEntry().keyAscii( "ChannelGroup", MapEntry.MapAction.ADD,
    elementList ));
elementList.clear();

innerElementList.add(EmaFactory.createElementEntry().ascii("DictionaryType",
    "DictionaryType::ChannelDictionary"));
innerElementList.add(EmaFactory.createElementEntry().ascii("RdmFieldDictionaryFileName",
    "./RDMFieldDictionary"));
innerElementList.add(EmaFactory.createElementEntry().ascii("EnumTypeDefFileName", "./enumtype.def));
```

```

innerMap.add(EmaFactory.createMapEntry().keyAscii( "Dictionary_1", MapEntry.MapAction.ADD,
    innerElementList));
innerElementList.clear();

elementList.add(EmaFactory.createElementEntry().map( "DictionaryList", innerMap ));
configMap.add(EmaFactory.createMapEntry().keyAscii( "DictionaryGroup", MapEntry.MapAction.ADD,
    elementList ));
elementList.clear();

```

3.5 Warm Standby Channel Group

WarmStandbyGroup is used only with the **Consumer**.

The **WarmStandbyGroup** contains a **WarmStandbyList**, which contains one or more **WarmStandbyChannel** entries (each uniquely identified by a **<Name .../>** entry). Each warm standby channel includes a set of warm standby channel parameters for specifying a starting active server, standby server(s), and a warm standby mode.

3.5.1 Generic XML Schema for WarmStandbyGroup

The top-level XML schema for the **WarmStandbyGroup** is as follows:

```

<WarmStandbyGroup>
  <WarmStandbyList>
    <WarmStandbyChannel>
      <Name value="VALUE"/>
      ...
    </WarmStandbyChannel>
  </WarmStandbyList>
</WarmStandbyGroup>

```

3.5.2 Warm Standby Channel Entry Parameters

Use the following parameters when configuring the **WarmStandbyGroup**.

PARAMETER NAME	TYPE	DEFAULT	NOTES
StartingActiveServer	EmaString	N/A	Specifies a starting server information to establish an initial connection. The name should have appropriate <WarmStandbyServerInfo> entry in the <WarmStandbyServerInfoGroup> .

Table 11: Warm Standby Channel Entry Parameters

PARAMETER NAME	TYPE	DEFAULT	NOTES
StandbyServerSet	EmaString	N/A	Specifies a comma-separated set of warm standby server information to establish connection(s). The name should have appropriate <WarmStandbyServerInfo> entry in the <WarmStandbyServerInfoGroup> .
WarmStandbyMode	Enumeration	LOGIN_BASED	Specifies the warm standby mode used by warm standby channel. Use enumeration values with Enterprise Message API programmatic configuration (for details, refer to Section 4.6). Available values include: • LOGIN_BASED(1) • SERVICE_BASED(2)

Table 11: Warm Standby Channel Entry Parameters

3.6 Warm Standby ServerInfo Group

WarmStandbyServerInfoGroup is used only with the **Consumer**.

The **WarmStandbyServerInfoGroup** contains a **WarmStandbyServerInfoList**, which contains one or more **WarmStandbyServerInfo** entries (each uniquely identified by a **<Name .../>** entry). Each warm standby server information is associated with a channel name and a set of preferred service names for service based warm standby.

When configuring for per-service based warm standby mode, the preferred service name set may be used to identify that channel as the starting channel for a given service.

3.6.1 Generic XML Schema for WarmStandbyServerInfoGroup

The top-level XML schema for the **WarmStandbyServerInfoGroup** is as follows:

```

<WarmStandbyServerInfoGroup>
  <WarmStandbyServerInfoList>
    <WarmStandbyServerInfo>
      <Name value="VALUE"/>
      ...
    </WarmStandbyServerInfo>
  </WarmStandbyServerInfoList>
</WarmStandbyServerInfoGroup>

```

3.6.2 Warm Standby ServerInfo Entry Parameters

Use the following parameters when configuring the **WarmStandbyServerInfoGroup**.

PARAMETER NAME	TYPE	DEFAULT	NOTES
Channel	EmaString	N/A	Specifies the channel that the warm standby channel should use. This channel must match the Name parameter from the appropriate <Channel> entry in the ChannelGroup configuration. For details on the <Channel> entry and default behaviors, refer to Section 3.4.
PerServiceNameSet	EmaString	N/A	Specifies a comma-separated set of preferred service name(s) for this server information. NOTE: This is used for the service based warm standby only.

Table 12: Warm Standby ServerInfo Entry Parameters

3.7 SessionChannel Group

Consumer applications use the **SessionChannelGroup** for routing market data item requests to multiple connections (refer to Chapter “Request Routing” of *Enterprise Message API Developers Guide*).

NiProvider applications use this group for session handling (refer to section “Non-Interactive Providers: Session Handling” in Chapter “Provider Classes” of *Enterprise Message API Developers Guide*).

The **SessionChannelGroup** contains a **SessionChannelList**, which contains one or more **SessionChannelInfo** entries, each uniquely identified by a **<Name .../>** entry. Each session channel info includes **ChannelSet** and/or **WarmStandbyChannelSet** for specifying a connection and reconnect parameters.

3.7.1 Generic XML Schema for SessionChannelGroup

The top-level XML schema for the **SessionChannelGroup** is as follows:

```
<SessionChannelGroup>
  <SessionChannelList>
    <SessionChannelInfo>
      <Name value="VALUE"/>
      ...
    </SessionChannelInfo>
  </SessionChannelList>
</SessionChannelGroup>
```

3.7.2 SessionChannel Entry Parameters

Use the following parameters when configuring the **SessionChannelGroup**.

PARAMETER NAME	TYPE	DEFAULT	NOTES
ChannelSet	String	N/A	Specifies a comma-separated set of channels names. Each listed channel name should have an appropriate <Channel> entry in the ChannelGroup . Channels in the set will be tried with each reconnection attempt until a successful connection is made. For further details refer to Section 3.4.8.
EnablePreferredHostOptions	Boolean	False	This is used to enable or disable the preferred host feature. This parameter may be altered at runtime using IOCTL. NOTE: This parameter must also be set to true to enable preferred host on this SessionChannel.
Name	String	N/A	Specifies the name of this Session Channel component. Name is required when creating a Session Channel component. This will be specified in the Consumer's SessionChannelSet. You can use any value for name.
PHDetectionTimeInterval	UInt64	0	Specifies time interval in second unit to switch over to a preferred host or WarmStandby group. A value of 0 indicates that the detection time interval is disabled. Precedence: if PHDetectionTimeSchedule is specified, this parameter is ignored. This parameter may be altered at runtime using IOCTL. This timer will start upon channel creation, and fire at that interval until either the channel is closed or if the interval is changed based on IOCTL. NOTE: This parameter has higher precedence than the PHDetectionTimeInterval parameter specified in the Consumer entry.
PHDetectionTimeSchedule	String	Empty string	Specifies Cron time format to switch over to a preferred host or WarmStandby group. PHDetectionTimeInterval, if specified, is used instead if this member is set to empty. This parameter may be altered at runtime using IOCTL. NOTE: This parameter has higher precedence than the PHDetectionTimeSchedule parameter specified in the Consumer entry.
PHFallbackWithInWSBGroup	Boolean	False	Specifies whether to fallback within a WarmStandby group instead of moving into a preferred WSB group. When setting PHFallbackWithInWSBGroup to false, the library closes all connections in the current WarmStandby group and attempts to connect to the preferred group. This cutover expects the starting active server to be available; if it is available, the library will connect to starting active and standby servers in the preferred group. This parameter may be altered at runtime using IOCTL. NOTE: This parameter has higher precedence than the PHFallbackWithInWSBGroup parameter specified in the Consumer entry.
PreferredChannelName	String	0	Specifies name of channel to set as preferred host. This parameter may be altered at runtime using IOCTL.
PreferredWSBChannelName	String	0	Specifies name of WarmStandby Group to set as preferred WarmStandby group. This parameter may be altered at runtime using IOCTL.

Table 13: SessionChannel Entry Parameters

PARAMETER NAME	TYPE	DEFAULT	NOTES
ReconnectAttemptLimit	int	-1	Specifies the maximum number of times the consumer and non-interactive provider attempt to reconnect to a channel when it fails. If set to -1, the consumer and non-interactive provider continually attempt to reconnect. NOTE: This parameter has higher precedence than the ReconnectAttemptLimit parameter specified in the Consumer entry.
ReconnectMaxDelay	int	5000	Sets the maximum amount of time the consumer and non-interactive provider wait (in milliseconds) before attempting to reconnect a failed channel. Refer also to the ReconnectMinDelay parameter. NOTE: This parameter has higher precedence than the ReconnectMaxDelay parameter specified in the Consumer entry.
ReconnectMinDelay	int	1000	Specifies the minimum amount of time the consumer and non-interactive provider wait (in milliseconds) before attempting to reconnect a failed channel. This waits time increases with each connection attempt, from ReconnectMinDelay to ReconnectMaxDelay. NOTE: This parameter has higher precedence than the ReconnectMinDelay parameter specified in Consumer entry.
WarmStandbyChannelSet	String	N/A	Specifies a comma-separated set of WarmStandby channels names. Each listed name should have an appropriate <WarmStandbyChannel> entry in WarmStandbyGroup. WarmStandby channels in the set will be moved to another WarmStandby channel when it failed to establish a connection for the entire WarmStandby channel. NOTE: - If both WarmStandbyChannelSet and ChannelSet are configured, then EMA uses the parameter that is configured in the WarmStandbyChannelSet before moving to ChannelSet. - When using both WarmStandby and ChannelSet, set reconnectAttemptLimit to a value other than default (-1) to avoid retrying servers within a WarmStandbyGroup indefinitely.

Table 13: SessionChannel Entry Parameters

3.8 Server Group

ServerGroup is used only with an **IProvider**.

The **ServerGroup** contains a **ServerList**, which contains one or more **Server** entries (each uniquely identified by a **<Name .../>** entry). Each channel includes a set of connection parameters for a specific connection or connection type.

There is no default server. If an Enterprise Message API application needs a specific server, you need to specify this in the appropriate **Consumer** or **IProvider** section.

- For details on the parameters you can use to configure the **Consumer** component, refer to Section 3.1.4.
- For details on the parameters you can use to configure the **IProvider** component, refer to Section 3.3.4.
- For a generic **ServerGroup** XML schema, refer to Section 3.8.1.
- For a list of parameters you can use in configuring **Server**, refer to Section 3.8.2.

3.8.1 Generic XML Schema for ServerGroup

The top-level XML schema for the **ServerGroup** is as follows:

```
<ServerGroup>
  <ServerList>
    <Server>
      <Name value="VALUE" />
      ...
    </Server>
  </ServerList>
</ServerGroup>
```

3.8.2 Server Entry Parameters

You can use the following parameters in any **<Server>** entry, regardless of the **ServerType**.

For additional information on how to set the **Server** connection type using the **ServerType** parameter, refer to Section 3.8.3.

PARAMETER NAME	TYPE	DEFAULT	NOTES
ConnectionMinPingTimeout	long	20000	Configures the minimum length of time (in milliseconds) to use as a timeout for a connected channel.
ConnectionPingTimeout	long	60000	Specifies the duration (in milliseconds) after which the Enterprise Message API terminates the connection if it does not receive communication or pings from the server.
CompressionThreshold	long	30	Sets the message size threshold (in bytes, the allowed value is 30-Integer.MAX_VALUE), above which all messages are compressed (thus individual messages might not be compressed). Different compression types have different behaviors and compression efficiency can vary depending on message size.

Table 14: Universal <Server> Parameters

PARAMETER NAME	TYPE	DEFAULT	NOTES
CompressionType	String	None	Specifies the Enterprise Message API's preferred type of compression. Compression is negotiated between the client and server: if the server supports the preferred compression type, the server will compress data at that level. Use strings with Enterprise Message API's programmatic configuration. For further details, refer to Section 4.6. Available values include: • None • ZLib • LZ4 NOTE: You can configure a server to force a particular compression type, regardless of client settings.
DirectWrite	int	0	Specifies whether to set the direct socket write flag when sending data on a channel. When the flag is set, every package is sent on the wire immediately on the submit call. If direct write is not set, the package might be placed into an internal queue which is later flushed onto the wire. Possible values are: • 0: Send data without the direct socket write flag. • 1: Send data with the direct socket write flag.
GuaranteedOutputBuffers	long	100	Specifies the number of guaranteed buffers (allocated at initialization time) available for use by each RsslChannel when writing data. Each buffer is created to contain maxFragmentSize bytes. For details on RsslChannel and maxFragmentSize, refer to the <i>Transport API Java Edition Developers Guide</i>.
HighWaterMark	long	6144	Specifies the upper buffer-usage threshold for the channel. Must be set explicitly in either in file or programmatic configuration.
InitializationTimeout	long	60	Specifies the time (in seconds) to wait for the successful initialization of a channel.
InterfaceName	String	""	Specifies a character representation of the IP address or hostname of the local network interface over which the Enterprise Message API sends and receives content. InterfaceName is for use in systems that have multiple network interface cards. If unspecified, the default network interface is used.
Name	String		Specifies the Server's name.
NumInputBuffers	long	100	Specifies the number of buffers used to read data. Buffers are sized according to maxFragmentSize. For details on RsslChannel and maxFragmentSize, refer to the <i>Transport API Java Edition Developers Guide</i>.
Port	String	14002	Specifies the port on the remote server to which the Enterprise Message API connects.

Table 14: Universal <Server> Parameters (Continued)

PARAMETER NAME	TYPE	DEFAULT	NOTES
ServerSharedSocket	long	0	Specifies whether the server allows socket sharing. Available values include: 0: The server does not allow socket sharing. (this is the default behavior) 1: The server allows socket sharing. For further details on ServerSharedSocket, refer to the <i>Transport API Java Edition Developers Guide</i>.
ServerType	String	RSSL_SOCKET	Specifies the type of channel or connection used to connect to the server. Calling the host function can change this field. For details on this event, refer to Section 4.4.2. Use strings with Enterprise Message API's programmatic configuration. For further details, refer to Section 4.6. Available values include RSSL_SOCKET (0), RSSL_ENCRYPTED (1), or RSSL_WEBSOCKET (7). RSSL_ENCRYPTED requires additional configuration provided via OmmIProviderConfig and programmatic configuration. For details, refer to Section 4.4.1. NOTE: Setting ServerType to RSSL_SOCKET or RSSL_WEBSOCKET has the same behavior. An open WebSocket request from the client side notifies the server to update the socket to a WebSocket connection type. The application is responsible for accepting or rejecting traffic based on a protocol that it intends to support.
SysRecvBufSize	long	65535	Specifies the size (in bytes) of the system's receive buffer for this channel. For exact, effective values, refer to your operating system documentation.
SysSendBufSize	long	65535	Specifies the size (in bytes) of the system's send buffer for this channel. For exact, effective values, refer to your operating system documentation.
TcpNodelay	long	1	Specifies whether to use Nagle's algorithm when sending data. Available values are: 0: Send data using Nagle's algorithm. 1: Send data without delay.

Table 14: Universal <Server> Parameters (Continued)

3.8.3 EMA Server Connection Types

Following is a sample snippet from the configuration file that shows how to set up the `ServerType` parameter:

```
<ServerType value="ServerType::RSSL_SOCKET"/>
```

The following table summarizes possible Server connection types and parameter values that you can use to set them.

SERVER CONNECTION TYPE	SERVERTYPE VALUE	SUBPROTOCOL LIST POSSIBILITIES
Unencrypted Socket	RSSL_SOCKET	The following possibilities apply: • By default, either websocket or socket incoming connections are accepted. • If the WsProtocols parameter is empty or mismatched with incoming connections, websocket connections are denied. • Socket connection logic ignores WsProtocols configuration and supports only RWF. • Default: WsProtocols value="tr_json2, rssl.rwf, rssl.json.v2"
Unencrypted Websocket	RSSL_WEBSOCKET	
Encrypted Socket	RSSL_ENCRYPTED	
Encrypted Websocket		

Table 15: Server Settings for Socket and Websocket Connection Types

3.8.4 Parameters for Use with ServerType RSSL_WEBSOCKET

You can use the following parameter when **ServerType** is set to **RSSL_WEBSOCKET**.

PARAMETER NAME	TYPE	DEFAULT	DESCRIPTION
WsProtocols	String	tr_json2, rssl.rwf, rssl.json.v2	Specifies a list of supported protocols in order of preference. Current protocols supported include rssl.json.v2 , rssl.rwf , and tr_json2 .

Table 16: RSSL_WEBSOCKET ServerType Parameter

Following is a sample snippet from the configuration file that shows how to set up the `ServerType` parameter:

```
<ServerType value="ServerType::RSSL_SOCKET"/>
```

The following table summarizes possible Server connection types and parameter values that you can use to set them.

SERVER CONNECTION TYPE	SERVERTYPE VALUE	SUBPROTOCOL LIST POSSIBILITIES
Unencrypted Socket	RSSL_SOCKET	The following possibilities apply: • By default, either websocket or socket incoming connections are accepted. • If the WsProtocols parameter is empty or mismatched with incoming connections, websocket connections are denied. • Socket connection logic ignores WsProtocols configuration and supports only RWF. • Default: WsProtocols value="tr_json2, rssl.rwf, rssl.json.v2"
Unencrypted Websocket	RSSL_WEBSOCKET	
Encrypted Socket	RSSL_ENCRYPTED	
Encrypted Websocket		

Table 17: Server Settings for Socket and Websocket Connection Types

3.9 Dictionary Group

The **DictionaryGroup** contains a **DictionaryList**, which contains one or more **Dictionary** components (each uniquely identified by a **<Name .../>** entry). Each **Dictionary** component defines parameters relating to how the dictionary is accessed.

3.9.1 Generic XML Schema for DictionaryGroup

The top-level XML schema for **DictionaryGroup** is as follows:

```
<DictionaryGroup>
  <DictionaryList>
    <Dictionary>
      <Name value="..." />
      ...
    </Dictionary>
  </DictionaryList>
</DictionaryGroup>
```

3.9.2 Dictionary Entry Parameters

Use the following parameters when configuring a **Dictionary** entry in the Enterprise Message API.

PARAMETER NAME	TYPE	DEFAULT	NOTES
DictionaryType	String	ChannelDictionary	Specifies the dictionary loading mode. Use strings with Enterprise Message API's programmatic configuration (for further details, refer to Section 4.6). Possible values are: FileDictionary (0): The Enterprise Message API loads the dictionaries from the files specified in the parameters RdmFieldDictionaryFileName and EnumTypeDefFileName. ChannelDictionary (1): The Enterprise Message API downloads dictionaries by requesting the dictionaries from the upstream provider.
EnumTypeDefFileName	String	./enumtype.def	Sets the location of the EnumTypeDef file.
EnumTypeDefItemName	String	RWFEnum	Sets the name of the EnumTypeDef item specified in the source directory InfoFilter.DictionariesProvided , and InfoFilter.DictionariesUsed elements.
Name	String		Sets a unique name for a Dictionary component in the DictionaryList .
RdmFieldDictionaryFileName	String	./RDMFieldDictionary	Sets the location of the RdmFieldDictionary .
RdmFieldDictionaryItemName	String	RWFFld	Sets the name of the RdmFieldDictionary item specified in the source directory InfoFilter.DictionariesProvided , and InfoFilter.DictionariesUsed elements.

Table 18: Dictionary Group Parameters

3.10 Directory Group

The **DirectoryGroup** contains a **DirectoryList**, which contains one or more **Directory** components (each uniquely identified by a **<Name .../>** entry). Each **Directory** component defines a list of **Service** components (which in turn define parameters that relate to the Service **InfoFilter** and **StateFilter**).

3.10.1 Generic XML Schema for Directory Entry

The top-level XML schema for **DirectoryGroup** is as follows:

```
<DirectoryGroup>
  <DefaultDirectory value="..." />
  <DirectoryList>
    <Directory>
      <Name value="..." />
      <Service>
        <Name value="..." />
        <InfoFilter>
          ...
        
```

```

        </InfoFilter>
        <StateFilter>
            ...
        </StateFilter>
    </Service>
    ...
</Directory>
...
</DirectoryList>
</DirectoryGroup>

```

3.10.2 Setting Default Directory

If you do not specify a **DefaultDirectory**, then the Enterprise Message API uses the first **Directory** component in the **DirectoryGroup**. However, you can specify a default directory by including the following parameter on a unique line inside **DirectoryGroup** but outside **DirectoryList**.

```
<DefaultDirectory value="VALUE"/>
```

3.10.3 Configuring a Directory in a DirectoryGroup

To configure a **Directory** component, add the following parameters (as appropriate) to the target directory in the XML Schema, each on a separate line:

PARAMETER	TYPE	DEFAULT	DESCRIPTION
Name	String	N/A	Specifies the name of this Directory component. Name is required when creating a Directory component. You can use any value for Name .
Service	Component Name	N/A	Specifies InfoFilter and StateFilter values for the given Service . NOTE: A Directory may contain several Service components.

Table 19: Directory Entry Parameters

3.10.4 Service Entry Parameters

The Service Entry resembles the RDM's Source Directory Domain payload. For further details, refer to the *Enterprise Message API Java Edition RDM Usage Guide*. The Enterprise Message API supports only the RDM entries **InfoFilter** and **StateFilter**. Use the following parameters when configuring a Service in the Enterprise Message API:

PARAMETER	TYPE	DEFAULT	DESCRIPTION
Name	String	N/A	Specifies the name of this Service component. You can use any value for Name .
InfoFilter	Component Name	N/A	Specifies InfoFilter values for the given Service . InfoFilter values set a filter on the types of information that the Enterprise Message API sends out.
StateFilter	Component Name	N/A	Specifies StateFilter values for the given Service . The Enterprise Message API sends StateFilter values to describe the service's state.

Table 20: Service Entry Parameters

3.10.5 InfoFilter Entry Parameters

The Enterprise Message API uses the following **InfoFilter** parameters to set filters on the types of information it sends over its services (as specified in the **EmaConfig.xml**).

PARAMETER	TYPE	DEFAULT	DESCRIPTION
ServiceId	int	N/A	Specifies the Service 's unique identifier. Available values include 0 - 65535.
Vendor	String	N/A	Specifies the name of the vendor that provides the service.
IsSource	int	0	Specifies whether the source of data sent on this service is its original publisher: 1: The service's data is provided directly by an original publisher 0: The service's data is a consolidation of multiple sources into a single service.
Capabilities	Component Name	N/A	A component that includes CapabilitiesEntry parameters, which define the message domain types that can be requested from the service. For details on the parameter used in this section, refer to Section 3.10.5.1.
ItemList	String	N/A	Specifies the name of the SymbolList that includes all items provided by this service.
DictionariesProvided	Component Name	N/A	A component that includes DictionariesProvidedEntry parameters, which define the dictionaries that the provider makes available. When specifying a dictionary, use the Dictionary 's component name whose *itemName entries are used in this Service's RDM DictionariesProvided entry. For details on the parameter used in this section, refer to Section 3.10.5.2.

Table 21: Source Directory Info Parameters

PARAMETER	TYPE	DEFAULT	DESCRIPTION
AcceptingConsumerStatus	int	1	Indicates whether a service can accept and process messages related to Source Mirroring. 0: The provider does not accept consumer status 1: The provider accept consumer status
DictionariesUsed	Component Name	N/A	A component that includes DictionariesUsedEntry parameters, which define the dictionaries that the provider uses. When specifying a dictionary, use the Dictionary 's component name whose *ItemName entries are used in this Service's RDM DictionariesUsed entry. For details on the parameter used in this section, refer to Section 3.10.5.3.
QoS	Component Name	Includes a single QoSEntry	A component that includes QoSEntry sections, with each QoSEntry section defining a QoS Timeliness and Rate supported by this Service. For details on the parameter used in this section, refer to Section 3.10.5.4.
SupportsQoSRange	int	0	Indicates whether the provider supports a QoS range when requesting an item. 0: The provider does not support a QoS Range. 1: The provider supports a QoS Range. For further details on using QoS ranges, refer to the <i>RDM Java Edition Usage Guide</i> .
SupportsOutOfBandSnapshots	int	For non-interactive provider: 0	Indicates whether the provider supports Snapshot requests after the OpenLimit has been reached: 0: The provider does not support snapshot requests. 1: The providers supports snapshot requests. For details on OpenLimit , refer to the <i>RDM Java Edition Usage Guide</i> .

Table 21: Source Directory Info Parameters (Continued)

3.10.5.1 Capabilities Entry Parameter

Use the **CapabilitiesEntry** parameter to configure the message domain type supported by the **Service** component:

PARAMETER	TYPE	DEFAULT	DESCRIPTION
CapabilitiesEntry	int or String	N/A	Specifies the message domain type supported by the Service component. Accepted names are listed in the EmaRdm interface. NOTE: You can set CapabilitiesEntry to be an RDM domain number or name (e.g. 6 or MMT_MARKET_PRICE).

Table 22: CapabilitiesEntry Parameter

3.10.5.2 DictionariesProvided Entry Parameter

Use the **DictionariesProvidedEntry** parameter to configure the dictionaries provided for the **Service's InfoFilter**:

PARAMETER	TYPE	DEFAULT	DESCRIPTION
DictionariesProvidedEntry	String	RWFFId for RdmFieldDictionaryItemName RWFFEnum for enumTypeDefItemName	Specifies the name of a Dictionary component from the DictionaryGroup section whose RdmFieldDictionaryItemName and enumTypeDefItemName parameters are used in this Service's RDM DictionariesProvided entry.

Table 23: DictionariesProvided Parameter

3.10.5.3 DictionariesUsed Entry Parameter

Use the **DictionariesUsedEntry** parameter to configure the types of dictionaries used by the **Service's InfoFilter**:

PARAMETER	TYPE	DEFAULT	DESCRIPTION
DictionariesUsedEntry	String	RWFFId for RdmFieldDictionaryItemName RWFFEnum for enumTypeDefItemName	Specifies the name of a Dictionary component from the DictionaryGroup section whose RdmFieldDictionaryItemName and enumTypeDefItemName are used in this Service's RDM DictionariesUsed entry.

Table 24: DictionariesUsedEntry Parameter

3.10.5.4 QoSEntry Section and Associated Parameters

Use a **QoSEntry** section to configure a specific QoS supported by the **Service's InfoFilter**. You can include multiple QoSEntry sections in a parent **QoS** section.

PARAMETER	TYPE	DEFAULT	DESCRIPTION
QoSEntry		N/A	QoSEntry is the name of a section that contains parameters specifying the Timeliness and Rate parameters for a given QoS. You can use multiple QoSEntry sections for a Service's InfoFilter .
Timeliness	int or String	Timeliness::Realtime	Specifies the QoS timeliness, which describes the age of the data (e.g., real time). NOTE: You can use numbers or names. Accepted names are listed in the OmmQos.Timeliness class.
Rate	int or String	Rate:TickByTick	Specifies the QoS rate, which is the rate of change for data sent over the Service . NOTE: You can use numbers or names. Accepted names are listed in the OmmQos.Rate class.

Table 25: QoSEntry Section and Associated Parameters

3.10.6 StateFilter Entry Parameters

Use the following parameters to configure the **Service's StateFilter** (as specified in **EmaConfig.xml**) which communicates the service state.

PARAMETER	TYPE	DEFAULT	DESCRIPTION
ServiceState	int	N/A	Specifies whether the service is up or down: 0: Service is down 1: Service is up
AcceptingRequests	int	For non-interactive provider: 0	Specifies whether the service accepts request messages: 0: The provider does not accept request messages. 1: The provider accepts request messages.
Status		Open / Ok / None / ""	Specifies a change in status to apply to all items provided by this service. The status only applies to items that received an OPEN/OK in a refresh or status message.

Table 26: StateFilter Parameters

3.10.7 Status Entry Parameters

Use the following parameters when configuring the **Service's StateFilter**:

PARAMETER	TYPE	DEFAULT	DESCRIPTION
StreamState	String	StreamState::Open	Specifies the state of the item stream. NOTE: Acceptable StreamState values are listed in the OmmState.StreamState class.
DataState	String	DataState::Ok	Specifies the state of the item data. NOTE: Acceptable DataState values are listed in the OmmState.DataState class.
StatusCode	String	StatusCode::None	Specifies the item status code. NOTE: Codes and their meanings are listed in the OmmState.StatusCode class.
StatusText	String	""	Specific StatusText regarding the current data and stream state. Typically used for informational purposes. StatusText has an encoded text with a maximum allowed length of 32,767 bytes.

Table 27: Status Entry Parameters

3.10.8 Setting Directory with Multiple Dictionaries Provided for IProvider

The following **EmaConfig.xml** example shows how to set up a Directory for the interactive provider. Note that the **DictionariesProvided** section of the Directory contains two **DictionariesProvided** entries. This feature allows the provider to supply multiple dictionaries to the consumer.

```
<?xml version="1.0" encoding="UTF-8"?>
<EmaConfig>

<IProviderGroup>
```

```

<DefaultIPProvider value="Provider_1"/>
<IPProviderList>
  <IPProvider>
    <Name value="Provider_1"/>
    <Server value="Server_1"/>
    <Directory value="Directory_1"/>
  </IPProvider>
</IPProviderList>
</IPProviderGroup>

<ServerGroup>
  <ServerList>
    <Server>
      <Name value="Server_1"/>
      <ServerType value="ServerType::RSSL_SOCKET"/>
      <CompressionType value="CompressionType::None"/>
      <Port value="14002"/>
    </Server>
  </ServerList>
</ServerGroup>

<DirectoryGroup>
  <DefaultDirectory value="Directory_1"/>
  <DirectoryList>
    <Directory>
      <Name value="Directory_1"/>
      <Service>
        <Name value="DIRECT_FEED"/>
        <InfoFilter>
          <ServiceId value="1"/>
          <Vendor value="TestLab"/>
          <DictionariesProvided>
            <DictionariesProvidedEntry value="Dictionary_1"/>
            <DictionariesProvidedEntry value="Dictionary_2"/>
          </DictionariesProvided>
          <DictionariesUsed>
            <DictionariesUsedEntry value="Dictionary_1"/>
          </DictionariesUsed>
          <Vendor value="company name"/>
          <IsSource value="0"/>
          <Capabilities>
            <CapabilitiesEntry value="MMT_MARKET_BY_ORDER"/>
            <CapabilitiesEntry value="MMT_DICTIONARY"/>
          </Capabilities>
          <QoS>
            <QoSEntry>
              <Timeliness value="Timeliness::RealTime"/>
              <Rate value="Rate::TickByTick"/>
            </QoSEntry>
          </QoS>
        </Service>
      </Directory>
    </DirectoryList>
  </DirectoryGroup>

```

```

        <ItemList value="#.itemlist"/>
        <SupportsOutOfBandSnapshots value="0"/>
    </InfoFilter>
    <StateFilter>
        <ServiceState value="1"/>
        <AcceptingRequests value="1"/>
    </StateFilter>
</Service>
</Directory>
</DirectoryList>
</DirectoryGroup>

<DictionaryGroup>
    <DictionaryList>
        <Dictionary>
            <Name value="Dictionary_1"/>
            <DictionaryType value="DictionaryType::FileDictionary"/>
            <RdmFieldDictionaryFileName value="./RDMFieldDictionary"/>
            <EnumTypeDefFileName value="./enumtype.def"/>
        </Dictionary>
        <Dictionary>
            <Name value="Dictionary_2"/>
            <DictionaryType value="DictionaryType::FileDictionary"/>
            <RdmFieldDictionaryFileName value="./RDMFieldDictionary_ID2"/>
            <EnumTypeDefFileName value="./enumtype_ID2.def"/>
            <RdmFieldDictionaryItemName value="RWFFld_ID2"/>
            <EnumTypeDefItemName value="RWFEnum_ID2"/>
        </Dictionary>
    </DictionaryList>
</DictionaryGroup>

</EmaConfig>

```

3.11 Global Configuration

The **GlobalConfig** group contains parameters that relate to the entire application and to all instances of Consumers, Interactive and Non-interactive Providers. They are set at the application start-up and remain the same during the application lifetime with some exceptions. Review each parameter to understand if it is a static configuration or may grow dynamically as required. For details on configuration precedence, refer to Section 4.1.

3.11.1 Generic XML Schema and Programmatic Configuration for GlobalConfig

The top-level XML schema for **GlobalConfig** is as follows:

```
<GlobalConfig>
  <ReactorMsgEventPoolLimit value="15"/>
  <ReactorChannelEventPoolLimit value="15"/>
  <WorkerEventPoolLimit value="15"/>
  <TunnelStreamMsgEventPoolLimit value="15"/>
  <TunnelStreamStatusEventPoolLimit value="15"/>
  <JsonConverterPoolsSize value="15"/>
  <WatchlistObjectsPoolLimit value="15"/>
  <SocketProtocolPoolLimit value="15"/>
</GlobalConfig>
```

The programmatic configuration for **GlobalConfig** is as follows:

```
Map configMap = EmaFactory.createMap();
ElementList elementList = EmaFactory.createElementList();

elementList.add(EmaFactory.createElementEntry().intValue( "ReactorMsgEventPoolLimit", 15));
elementList.add(EmaFactory.createElementEntry().intValue( "ReactorChannelEventPoolLimit", 15));
elementList.add(EmaFactory.createElementEntry().intValue( "WorkerEventPoolLimit", 15));
elementList.add(EmaFactory.createElementEntry().intValue( "TunnelStreamMsgEventPoolLimit", 15));
elementList.add(EmaFactory.createElementEntry().intValue( "TunnelStreamStatusEventPoolLimit", 15));
elementList.add(EmaFactory.createElementEntry().intValue( "JsonConverterPoolsSize", 15));
elementList.add(EmaFactory.createElementEntry().intValue( "WatchlistObjectsPoolLimit", 15));
elementList.add(EmaFactory.createElementEntry().intValue( "SocketProtocolPoolLimit", 15));

configMap.add(EmaFactory.createMapEntry().keyAscii( "GlobalConfig", MapEntry.MapAction.ADD,
  elementList ));
elementList.clear();
```

For details on programmatic configuration, refer to Section 4.6.

NOTE: The **GlobalConfig** parameters should be specified if the needed value is different from the default one.

3.11.2 Global Parameters

Enterprise Message API uses global parameters listed in the following table. All parameters are optional.

PARAMETER	TYPE	DEFAULT	DESCRIPTION
EmaObjectManagerComplexTypeLimit	int	-1	Specifies the size of the pools that keep EMA complex type objects (ElementList , FieldList , etc.). The value of -1 means that the pools have no limit.
EmaObjectManagerDataTypeLimit	int	-1	Specifies the size of the pools that keep EMA data type objects. The value of -1 means that the pools have no limit.
EmaObjectManagerEtaObjectsLimit	int	-1	Specifies the size of the pools that keep necessary objects from the ETA layer. The value of -1 means that the pools have no limit.
EmaObjectManagerMsgTypeLimit	int	-1	Specifies the size of the pools that keep EMA message type objects (RefreshMsg , UpdateMsg , etc.). The value of -1 means that the pools have no limit.
EmaObjectManagerSessionObjectsLimit	int	-1	Specifies the size of the pools that keep EMA objects associated with application sessions. The value of -1 means that the pools have no limit.
JsonConverterPoolsSize	int	10	Specifies the maximum number of objects available to JsonFactory , used by the RWF/JSON converter library to perform conversion. These pools are common to all instances of OMMConsumer/OMMProvider and must be preconfigured to account for all instances created by the application. This is pre-allocated regardless of the connection type initially specified. If additional objects are required beyond the specified value of the pools, objects are added to the pools, thereby increasing the size of the pools.
ReactorChannelEventPoolLimit	int	-1	Specifies the maximum number of events in the ReactorChannelEvent pool. When the limit is reached, events are created outside of the pool and garbage collected later. If this value is negative (such as the default setting), then the number of events is unlimited.
ReactorMsgEventPoolLimit	int	-1	Specifies the maximum number of events in the ReactorMsgEvent pool. When the limit is reached, events will be created outside of the pool and garbage collected later. If this value is negative (such as the default setting), then the number of events is unlimited.
SocketProtocolPoolLimit	int	-1	Specifies the maximum number of objects in the Transport pool. The Transport pools and reuses TCP and Websocket protocol objects for internal Transport Channels. If the limit is set, the Transport pools only a limited number of Channel and Server objects, and any additional objects will be garbage collected when the OmmConsumer/OmmProvider objects are destroyed. If the value is negative (such as the default setting), then the number of pooled objects is unlimited.
TunnelStreamMsgEventPoolLimit	int	-1	Specifies the maximum number of events in the TunnelStreamMsgEvent pool. When the limit is reached, events are created outside of the pool and will be garbage collected later. If value is negative (such as the default setting), then the number of events is unlimited.
TunnelStreamStatusEventPoolLimit	int	-1	Specifies the maximum number of events in the TunnelStreamStatusEvent pool. When the limit is reached, events are created outside of the pool and garbage collected later. If the value is negative, the amount of number is unlimited.

Table 28: Global Configuration Parameters

PARAMETER	TYPE	DEFAULT	DESCRIPTION
WatchlistObjectsPoolLimit	int	-1	Specifies the maximum number of events in the Watchlist pool. When the limit is reached, events are created outside of the pool and garbage collected later. If the value is negative (such as the default setting), the number of events is unlimited.
WorkerEventPoolLimit	int	-1	Specifies the maximum number of events in the WorkerEvent pool. When the limit is reached, events are created outside of the pool and garbage collected later. If the value is negative (such as the default setting), then the number of events is unlimited.

Table 28: Global Configuration Parameters(Continued)

NOTE: The **Global configuration** contains parameters that control the limits of the **Global EmaObjectManager** pools. The same parameters are also available in per-instance configurations for Consumer, Provider, and NiProvider, where they apply only to the object pools managed by the **EmaObjectManager** of that specific instance. The **Global EmaObjectManager** is separate from the per-instance managers and is used when objects must be shared across instances and not bound to a specific instance, such as during message cloning. Within a single application, all Consumer, Provider, and NiProvider instances share the **Global EmaObjectManager** while each maintains its own per-instance **EmaObjectManager** with independently configured pool limits.

4 Enterprise Message API Configuration Processing

4.1 Overview and Configuration Precedence

The Enterprise Message API configuration is determined by hard-coded behaviors, customized behaviors as specified in a configuration file (i.e., **EmaConfig.xml**), programmatic changes, and other internal processing. All of these vectors affect Enterprise Message API's configuration as used by application components. The Enterprise Message API merges configuration parameters specified from all vectors with the following precedence: function calls, programmatic configuration, file configuration, and finally the default configuration. For example, if parameters are specified in both function calls and the programmatic configuration, the function call configuration takes precedence.

4.2 Default Configuration

4.2.1 Default Consumer Configuration

Each Enterprise Message API consumer-type application must eventually instantiate an **OmmConsumer** object. Constructors for **OmmConsumer** require a **OmmConsumerConfig** object. The **OmmConsumerConfig** constructor can read and process an optional XML file, which applications can use to modify Enterprise Message API's default consumer behavior. By default this file is named **EmaConfig.xml** and stored in the working directory. For details on using non-default names and directories for your XML configuration file, refer to Section 4.3.1.2.

The Enterprise Message API provides a hard-coded configuration for use whenever an **OmmConsumerConfig** object is instantiated without a configuration file (such as **EmaConfig.xml**) in the run-time environment. The resulting configuration is created by taking the defaults from the various configuration groups. For example, the default (hard-coded) behavior for a **Channel** adheres to the following configuration:

- **ChannelType** value="RSSL_SOCKET"
- **CompressionType** value="None"
- **TcpNoDelay** value="1"
- **Host** value="localhost"
- **Port** value="14002"

Note that unlike the Enterprise Message API's default behavior of choosing the first **Consumer** component in the **ConsumerList**, Enterprise Message API applications will not choose the first **Channel** or **Dictionary** in their respective lists. Instead, if an application wants to use a specific channel or dictionary configuration, the application must explicitly configure it in the appropriate **Consumer** section of the XML file.

4.2.2 Default Provider Configurations

Each Enterprise Message API provider-type application must eventually instantiate an **OmmProvider** object. Constructors for **OmmProvider** require a **OmmProviderConfig** object. The **OmmProviderConfig** constructor can read and process an optional XML file, which applications can use to modify the Enterprise Message API's default provider behavior. By default this file is named **EmaConfig.xml** and stored in the working directory. For details on using non-default names and directories for your XML configuration file, refer to Section 4.3.1.2.

The Enterprise Message API provides a hard-coded configuration for use whenever an **OmmProviderConfig** object is instantiated without an **EmaConfig.xml** file in the run-time environment. The resulting Enterprise Message API configuration is created by taking the defaults from the various configuration groups.

4.2.2.1 Example: Default Channel Behavior (NiProvider)

The default (hard-coded) behavior for a **Channel** adheres to the following configuration:

- **ChannelType** value="RSSL_SOCKET"
- **CompressionType** value="None"
- **TcpNoDelay** value="1"
- **Host** value="localhost"
- **Port** value="14003"

Note that unlike the Enterprise Message API's default behavior of choosing the first **NiProvider** component in the **NiProviderList**, Enterprise Message API applications will not choose the first or **Channel** in their respective lists. Instead, if an application wants to use a specific channel or dictionary configuration, the application must explicitly configure it in the appropriate **NiProvider** section of the XML file.

4.2.2.2 Example: Default Server Behavior (IProvider)

The default (hard-coded) behavior for a **Server** adheres to the following configuration:

- **ServerType** value="RSSL_SOCKET"
- **CompressionType** value="None"
- **TcpNoDelay** value="1"
- **Port** value="14002"

Note that unlike the Enterprise Message API's default behavior of choosing the first **IProvider** component in the **IProviderList**, Enterprise Message API applications will not choose the first or **Server** in their respective lists. Instead, if an application wants to use a specific server or dictionary configuration, the application must explicitly configure it in the appropriate **IProvider** section of the XML file.

4.3 Processing EMA's XML Configuration File

The LSEG Real-Time SDK package installs a default configuration file named **EmaConfig.xml** into the Enterprise Message API's working directory. By default, the Enterprise Message API looks for a configuration file with this name in the working directory. If you want to use a different name for your configuration file, and/or store the file in a directory other than the working directory, you must specify this filename and/or directory in your configuration object. For further details on using the configuration object, how it functions as regards paths and filenames, and how the Enterprise Message API determines its configuration, refer to Section 4.3.1.

Except for the parameters **DefaultConsumer** and **DefaultNiProvider**, you must wrap all other elements defined in the Enterprise Message API's configuration file in a component definition (i.e., **Consumer**, **NiProvider**, **Channel**, or **Dictionary**) otherwise the Enterprise Message API ignores the element. This section includes some examples that illustrate this requirement.

4.3.1 Reading the Configuration File

NOTE: The following section uses Consumer objects (i.e., **OmmConsumer** and **OmmConsumerConfig**) to illustrate how the Enterprise Message API checks for a configuration file, and if one exists, how the Enterprise Message API starts to process it.

The **OmmConsumer** constructor expects an **OmmConsumerConfig** object. By default, **OmmConsumerConfig** searches its working directory for a configuration file by the name of **EmaConfig.xml**. However, if you store your configuration file elsewhere on the system, or use a custom filename, you can include an argument with the configuration object to specify the alternate path and/or name of your configuration file.

4.3.1.1 Using EmaConfig.xml in CLASSPATH or in the Working Directory

If **OmmConsumerConfig** lacks an argument, the application attempts to open a configuration file named **EmaConfig.xml**, which must be located in either the **CLASSPATH** root directories or in the current working directory. By precedence, the Enterprise Message API uses the **EmaConfig.xml** from the **CLASSPATH** root directories; if one does not exist in the **CLASSPATH**, then the Enterprise Message API uses the **EmaConfig.xml** from the current working directory. If the Enterprise Message API does not find an **EmaConfig.xml**, the application uses the default configuration. Additionally, if the **EmaConfig.xml** file is empty or contains malformed XML, the application uses the default configuration. For details on the default configuration, refer to Section 4.2.

For example, to use an **EmaConfig.xml** stored in CLASSPATH or in the working directory, have the application create an **OmmConsumerConfig** object (for details on this object, refer to the *Enterprise Message API Java Developers Guide*) and pass it to the **OmmConsumer** object as follows:

```
OmmConsumerConfig config = EmaFactory.createOmmConsumerConfig();

consumer = EmaFactory.createOmmConsumer(config);
```

For complete details, refer to the example *example100__MarketPrice__Streaming* included with the LSEG Real-Time SDK.

4.3.1.2 Using a Custom Filename and/or Directory

If you include a path with `OmmConsumerConfig`, the application creates a filename from the argument and attempts to open a file with that name, as follows:

- If the argument represents only a directory, the Enterprise Message API appends **EmaConfig.xml** to the argument and verifies whether **EmaConfig.xml** exists in the specified directory.
- If the argument represents a directory and filename, the Enterprise Message API verifies whether the specified file exists.
- If the specified file does not exist, the application throws an `IceException`, which indicates the specified path and the current working directory.
- If the argument represents neither a file nor a directory, an `IceException` is thrown.

At this point, the application attempts to create an XML configuration from the filename. If the attempt fails, the application throws an `IceException`.

If you want to specify a custom path and filename, have the application create an `OmmConsumerConfig` object with the path and filename in the argument (for details on this object, refer to the *Enterprise Message API Java Developers Guide*) and pass it to the `OmmConsumer` object as follows (where **PATH** is the alternate path and/or filename you want to use for your configuration file):

```
OmmConsumerConfig config = EmaFactory.createOmmConsumerConfig(PATH);

consumer = EmaFactory.createOmmConsumer(config);
```

For an example of how to specify a custom configuration file name, refer to *Example 111* (**example111__MarketPrice__UserSpecifiedFileConfig**) included with the package.

4.3.2 Use of the Correct Order in the XML Schema

In the following configuration file snippet (only those parts needed for the example are included), the application creates a consumer with a **Name of Consumer_1**.

```
<ConsumerGroup>
  <ConsumerList>
    <Consumer>
      <Name value="Consumer_1"/>
    </Consumer>
  </ConsumerList>
</ConsumerGroup>
```

Now assume that the following was not included in the XML configuration:

```
<Directory>
  <Name value="Directory_1"/>
```

In this case, the Enterprise Message API application relies on its hard-coded behavior.

However, if the snippet is configured in either of the following configurations, the Enterprise Message API application reverts to its default behaviors because the parameters are not in the correct order (i.e., the **Name** parameter needs to be contained in a **Directory** component entry):

- Configuration 1:

```
<DirectoryGroup>
  <Name value="Name"/>
  <DirectoryList>
    ...
```

- Configuration 2:

```
<DirectoryGroup>
  <DirectoryList>
    <Name value="Name"/>
    <Directory>
      ...
```

4.3.3 Processing the Consumer “Name”

The Enterprise Message API is hard-coded to use a default consumer of **EmaConsumer**. However, you can change this by using the configuration file (e.g., **EmaConfig.xml**). When you use the XML file, the default **Consumer Name** is either specified by the **DefaultConsumer** element, or if this parameter is not set, then the Enterprise Message API application will default to the name of the first Consumer component.

- If **DefaultConsumer** uses an invalid name (i.e., no **Consumer** components in the XML file use that name), the Enterprise Message API throws an exception indicating that **DefaultConsumer** is invalid.
- If the configuration file has no **Consumer** components, the Enterprise Message API application uses **EmaConsumer**.

4.3.4 Processing the Provider “Name”

The Enterprise Message API is hard-coded to use a default non-interactive provider of **EmaProvider**. However, you can change this by using the configuration file (e.g., **EmaConfig.xml**). When you use the XML file, the default **Provider Name** is either specified by the **DefaultProvider** element, or if this parameter is not set, then the Enterprise Message API application will default to the name of the first non-interactive provider component.

- If **DefaultProvider** uses an invalid name (i.e., no **Provider** components in the XML file use that name), the Enterprise Message API throws an exception indicating that **DefaultProvider** is invalid.
- If the **EmaConfig.xml** has no **Provider** components, the Enterprise Message API application uses **EmaProvider**.

4.4 Configuring Enterprise Message API Using Method Calls

From an application standpoint, instantiating `OmmConsumerConfig` and `OmmNiProviderConfig` objects creates the initial configuration from the Enterprise Message API's XML configuration file (if one exists). Certain variables can then be altered via method calls on the `OmmConsumerConfig` and `OmmProviderConfig` objects.

NOTE: Method calls override any settings in a configuration XML file.

4.4.1 EMA Configuration Method Calls

4.4.1.1 OmmConsumerConfig Class Method Calls

You can use the following method calls in an Enterprise Message API **Consumer** application:

METHOD	DESCRIPTION
<code>addAdminMsg(ReqMsg)</code>	Populates part of or all of the login request message, directory request message, or dictionary request message according to the specification discussed in the <i>Enterprise Message API LSEG Data & Analytics Domain Model (RDM) Usage Guide</i> specific to the programming language you use to override the default administrative request. Application may call multiple times prior to initialization.
<code>addServiceList(ServiceList serviceList)</code>	Adds the indicated Service List to <code>OmmConsumerConfig</code> . The Service List name can be specified in <code>reqMsg.ServiceListName</code> to indicate that the request is for any service in the Service List. For more information, see section "Service List" in chapter "Request Routing" in the <i>Enterprise Message API Developers Guide</i> .
<code>applicationId(String)</code>	Sets the applicationId variable. applicationId has no default value.
<code>audience(String)</code>	Optional. Used only with Version 2 <code>oAuthClientCredential</code> with JWT. Sets the audience claim for the JWT.
<code>channelType(ConnectionTypes)</code>	Optional. Specifies the channel type used by the current consumer. Use EmaConfig.ConnectionType to set allowed connection type.
<code>clear()</code>	Clears existing content from the <code>OmmConsumerConfig</code> object.
<code>clientId(String)</code>	Required. Specifies an authentication parameter. - Version 1 Authentication: a unique ID for application used to make an authentication request. - Version 2 Authentication: a unique ID provisioned as part of Service Account used to make an authentication request. For details on Version 1 and Version 2 Authentication, refer to the <i>Enterprise Message API Developers Guide</i> , Section "Consuming Data from the Cloud".
<code>clientJwk(String)</code>	Required for Version 2 <code>oAuthClientCredential</code> with JWT. Sets the JWK formatted private key used to create the JWT. The JWT is used to authenticate with the RDP token service.
<code>clientSecret(String)</code>	Sets the client secret. Optional for Version 1 <code>oAuthPasswordGrant</code> authentication. Required for Version 2 <code>oAuthClientCredential</code> authentication and provisioned as part of Service Account.
<code>config(Data)</code>	Passes in the consumer's programmatic configuration.

Table 29: OmmConsumerConfig Class Method Calls

METHOD	DESCRIPTION
consumerName(String)	Sets the consumer name, which is used to select a specific consumer as defined in the Enterprise Message API's configuration. If a consumer does not exist with that name, the application throws an exception.
dataDictionary(DataDictionary, boolean)	Optional. Specifies the DataDictionary object with a mandatory bool or flag. If flag is true, the DataDictionary object will be copied into API space; otherwise, it will be passed in as a reference. Overrides DataDictionary object provided via EmaConfig.xml or programmatic configuration.
encryptedProtocolType(EncryptedProtocol Types)	Optional. Specifies the encrypted protocol type used by the current consumer. Use EmaConfig.EncryptedProtocolType to set allowed encrypted protocol type.
host(String)	Sets the host and port parameters. For details, refer to Section 4.4.2. Sample value: "localhost:14002".
negativeUpdateTypeFilter(ulong)	Specifies a filter on <i>types</i> of updates NOT desired. The value is a bitmask, where each bit indicates a specific update type to exclude. Possible bit values are defined in the EmaRdm class in com.refinitiv.ema.rdm package. The NegativeUpdateTypeFilter value is sent to server in the LOGIN request message to request <i>exclusion</i> of specified update types. If this is specified with UpdateTypeFilter , the server will honor only one of the filters per decode order.
operationModel(OperationModel)	Optional. Sets the operation model to either of these: OperationModel.API_DISPATCH (default) OperationModel.USER_DISPATCH
password(String)	Required for Version 1 oAuthPasswordGrant authentication. Specifies the password used together with the username to obtain the access token.
position(String)	Sets the position variable. position has no default value.
restProxyDomain	Optional. Specifies the domain of the user to authenticate to the proxy server for Rest requests: service discovery and auth token service. Needed for NTLM or for Negotiate/Kerberos or for Kerberos authentication protocols.
restProxyHostName	Optional. Specifies the address or host name of the proxy server for REST requests: service discovery and auth token service.
restProxyKrb5ConfigFile	Optional. Specifies the complete path of the Kerberos5 configuration file (krb5.ini or krb5.conf, or custom file). Needed to Negotiate/Kerberos and Kerberos authentications for the proxy connection.
restProxyLocalHostName	Optional. Specifies the local hostname of the client. Needed for NTLM authentication protocol only.
restProxyPasswd	Optional. Specifies the password to authenticate to the proxy server for REST requests: service discovery and auth token service. Needed for all authentication protocols.
restProxyPort	Optional. Specifies the port number of the proxy server for REST requests: service discovery and auth token service.
restProxyUserName	Optional. Specifies the user name to authenticate to the proxy server for REST requests: service discovery and auth token service. Needed for all authentication protocols.
ServiceDiscoveryUrl(String)	Optional. Specifies a URL to override the default for the RDP service discovery to get global endpoints. Default value is https://api.refinitiv.com/streaming/pricing/v1/ .

Table 29: OmmConsumerConfig Class Method Calls(Continued)

METHOD	DESCRIPTION
takeExclusiveSignOnControl(bool)	Optional. Specifies exclusive sign on control to force sign-out of other applications using the same credentials. Default value is True .
tokenScope(String)	Optional for Version 1 and Version 2 authentication. Specifies token scope to override the default for limiting the scope of generated token from the token service. Defaults to trapi.streaming.pricing.read .
tokenServiceUrlV1(String)	Optional. Specifies a URL to override the default for token service V1 oAuthPasswordGrant to perform authentication to get access and refresh tokens. Default value is https://api.refinitiv.com/auth/oauth2/v1/token .
tokenServiceUrlV2(String)	Optional. Specifies a URL to override the default for token service V2 oAuthClientCredentials to perform authentication to get access and refresh tokens. Default value is https://api.refinitiv.com/auth/oauth2/v2/token .
tunnelingCredentialUserName(String)	Optional. Specifies the username for an authenticated proxy.
tunnelingCredentialPasswd(String)	Optional. Specifies the password for an authenticated proxy.
tunnelingCredentialDomain(String)	Optional. Specifies the domain for an NTLM or Kerberos authenticated proxy.
tunnelingCredentialKRB5ConfigFile(String)	Optional. Specifies the path to the Kerberos5 configuration file (krb5.ini or krb5.conf , or custom file). Needed for Kerberos proxy authentication.
tunnelingCredentialLocalHostName(String)	Optional. Specifies the local Host Name of the client. Needed for NTLM authentication protocol only.
tunnelingKeyStoreType(String)	Optional. Specifies type of the key store for certificate file. This is used for both authenticated proxy interactions and encrypted connections. This is optional.
tunnelingKeyStoreFile(String)	Optional. Specifies the key store file that will be used for both authenticated proxy interactions and encrypted connections. This is optional. By default, Enterprise Message API will load the JVM's certificate store location.
tunnelingKeyStorePasswd(String)	Optional. Specifies the key store password. This is required if a key store is specified.
tunnelingSecurityProtocol(String)	Optional. Specifies the cryptographic protocols to be used for an Encrypted connection. TLS 1.2 and TLS 1.3 are supported.
tunnelingSecurityProtocolVersions (String[])	Optional. Specifies the versions of the configured security protocol. For TLS, this means you can add your selected version(s) to this string array as "1.2" and "1.3". Defaults to both "1.2" and "1.3".
tunnelingSecurityProvider(String)	Optional. Specifies the cryptographic security provider for the JVM. Default is SunJSSE. RTSDK library also supports Conscrypt security provider. To choose this provider, specify "Conscrypt" as the argument to this method.
tunnelingKeyManagerAlgorithm(String)	Optional. Specifies the key manager algorithm. Default is SunX509.
tunnelingTrustManagerAlgorithm(String)	Optional. Specifies the trust manager algorithm. Default is PKIX.
tunnelingLibCryptoName(String)	Optional. Specifies the name of the libcrypto.so shared library for Encrypted connections. See <i>Enterprise Message API Java Edition Developers Guide</i> for details on default libcrypto names per platform and OpenSSL version.
tunnelingLibSslName(String)	Optional. Specifies the name of the libssl.so shared library for Encrypted connections. See the <i>Enterprise Message API Java Edition Developers Guide</i> for details on default libssl names per platform and OpenSSL version.
tunnelingObjectName(String)	Optional. Specifies the object name to pass along with the underlying URL in HTTP and HTTPS connection messages.

Table 29: OmmConsumerConfig Class Method Calls(Continued)

METHOD	DESCRIPTION
tunnelingProxyHostName(String)	Optional. Specifies the host name of an HTTP Proxy for any Socket, Websocket, or Encrypted connections.
tunnelingProxyPort(String)	Optional. Specifies the port number of the proxy server to connect to for an HTTP connection.
updateTypeFilter(long)	Specifies a filter on <i>types</i> of updates desired. The value is a bitmask, where each bit indicates a specific update type. Possible bit values are defined in the EmaRdm class in com.refinitiv.ema.rdm package. The UpdateTypeFilter value is sent to server in the LOGIN request message to request <i>inclusion</i> of requested update types. If this is specified with NegativeUpdateTypeFilter , the server will honor only one of the filters per decode order.
username(String)	Required for Version 1 oAuthPasswordGrant authentication. Specifies the username used to obtain the authentication token from the Delivery Platform and provisioned with Machine Account. Optional otherwise. Sets username; if not specified, username is extracted from run-time environment.
wsbChangeEventInfo(boolean)	Specifies whether Enterprise Message API provides Warm Standby change information with Login StatusMsg notifications. The default is false.

Table 29: OmmConsumerConfig Class Method Calls(Continued)

4.4.1.2 OmmIProviderConfig Class Method Calls

You can use the following method calls in an Enterprise Message API **IProvider** application. For further details on variables, refer to the *Enterprise Message API Java RDM Usage Guide*.

METHOD	DESCRIPTION
addAdminMsg(RefreshMsg)	Optional. Populates the entirety of the initial directory refresh message according to the specification discussed in the <i>Enterprise Message API Java Edition RDM Usage Guide</i> . Supports Directory domain only.
adminControlDictionary(AdminControl)	Optional. Specifies whether API or user controls responding to Dictionary requests. Default is API_CONTROL .
adminControlDirectory(AdminControl int)	Specifies whether the API or the user controls the sending of Directory refresh messages. Default is API_CONTROL .
clear()	Clears existing content from the OmmIProviderConfig object.
config(Data)	Passes in the provider's programmatic configuration.
keyManagerAlgorithm(String)	Optional. Specifies the security provider.
keystoreFile(String)	Optional. The file name of the keystore file.
keystorePasswd(String)	Optional. The keystore password.
keystoreType(String)	Optional. The keystore type. Default is JKS .
operationModel(OperationModel)	Optional. Specifies the operation model. Default is API Dispatch: ApiDispatchEnum .
port(EmaString)	Optional. Specifies a port. Default is 14002 .
providerName(String)	This name identifies configuration section to be used by an OmmIProvider instance.

Table 30: OmmIProviderConfig Class Method Calls

METHOD	DESCRIPTION
securityProtocol(String)	Optional. Specifies the security protocol.
securityProtocolVersions(String[])	Optional. Specifies the versions of the configured security protocol. For TLS, this means you can add your selected version(s) to this string array as “1.2” and “1.3”. Defaults to both “1.2” and “1.3”.
securityProvider(String)	Optional. Specifies the security provider. RTSDK library supports SunJSSE and Conscrypt security providers. To choose a provider, specify either “SunJSSE” or “Conscrypt” string as an argument to this method.
trustManagerAlgorithm(String)	Optional. Specifies the trust manager algorithm.

Table 30: **OmmIPProviderConfig** Class Method Calls (Continued)

4.4.1.3 OMMNiProviderConfig Class Method Calls

You can use the following method calls in an Enterprise Message API **NiProvider** application. For further details on variables, refer to the *Enterprise Message API Java Edition RDM Usage Guide*.

METHOD	DESCRIPTION
addAdminMsg(const RefreshMsg&)	Optional. Specifies an administrative refresh message to override the default administrative refresh. Supports Directory domain only.
addAdminMsg(RequestMsg)	Used only with NiProvider . Optional. Populates part of or all of the login request message according to the specification discussed in the <i>Enterprise Message API Java Edition RDM Usage Guide</i> . Supports Login domain only.
adminControlDirectory(AdminControl)	Optional. Specifies whether the API or the user controls the sending of Directory refresh messages. Available values include: OmmNiProviderConfig::ApiControlEnum (which is the default) OmmNiProviderConfig::UserControlEnum For details on control models, refer to OmmNiProviderConfig.h .
applicationId(const String&)	Optional. Specifies the authorization application identifier set in login request attribute. Must be unique for each application. No default value. Range 257 to 65535 is available for site-specific use. Range 1 to 256 is reserved.
channelType(ConnectionTypes)	Optional. Specifies the channel type used by the current provider. Use EmaConfig.ConnectionType to set allowed connection type.
clear()	Clears existing content from the OmmProviderConfig object.
config(const Data&)	Passes in the NiProvider’s programmatic configuration.
encryptedProtocolType(EncryptedProtocol Types)	Optional. Specifies the encrypted protocol type used by the current provider. Use EmaConfig.EncryptedProtocolType to set allowed encrypted protocol type.
host(const String&)	Optional. Specifies a hostname and port. By default, it is set to localhost:14003 .
instanceId(const String&)	Optional. Specifies the instance identifier. Can be any ASCII string, e.g. “Instance1”.
operationModel(OperationModel)	Optional. Specifies the operation model. Default is API Dipatch: ApiDispatchEnum .

Table 31: **OmmNiProviderConfig** Class Method Calls

METHOD	DESCRIPTION
<code>password(const String&)</code>	Specifies password. Overrides a value specified in Login domain via the addAdminMsg(const ReqMsg&) method.
<code>position(const String&)</code>	Specifies position in login request attribute.
<code>providerName(const String&)</code>	This name identifies configuration section to be used by OmmNiProvider instance.
<code>tunnelingCredentialDomain(String)</code>	Optional. Specifies the domain for tunneling connection.
<code>tunnelingCredentialLocalHostName(String)</code>	Specifies the client local host name used for tunneling connection. Needed for NTLM authentication only.
<code>tunnelingCredentialPasswd(String)</code>	Optional. Specifies the password for tunneling connection.
<code>tunnelingCredentialUserName(String)</code>	Optional. Specifies the userName for tunneling connection.
<code>tunnelingKeyManagerAlgorithm(String)</code>	Optional. Specifies a Java key manager algorithm for tunneling connection.
<code>tunnelingKeyStoreFile(String)</code>	Specifies the key storing file for tunneling connection. The keytool JDK utility creates this file.
<code>tunnelingKeyStoreType(String)</code>	Optional. Specifies the type of key store for tunneling connection. Default is JKS .
<code>tunnelingObjectName(const EmaString&)</code>	Specifies the object name to pass along with the underlying URL in HTTP and HTTPS connection messages.
<code>tunnelingProxyHostName(const EmaString&)</code>	Optional. Specifies the address or host name of the proxy server to connect to for an HTTP or HTTPS connection.
<code>tunnelingProxyPort(const EmaString&)</code>	Optional. Specifies the port number of the proxy server to connect to for an HTTP or HTTPS connection.
<code>tunnelingSecurityProtocol(intString)</code>	Optional. Specifies the cryptographic protocols to be used for an Encrypted connection. The highest TLS version supported by API will be selected by the Rssl API first, then it will roll back if the encryption handshake fails. The protocol supports TLS v1.2 and TLS v.1.3. Use OmmNiProviderConfig::EncryptedProtocolTypes flags to set allowed protocols.
<code>tunnelingSecurityProtocolVersions(String[])</code>	Optional. Specifies the versions of the configured security protocol. For TLS, this means you can add your selected version(s) to this string array as “1.2” and “1.3”. Defaults to both “1.2” and “1.3”.
<code>tunnelingSecurityProvider(String)</code>	Optional. Specifies a Java cryptography package provider for tunneling connection. RTSDK library supports SunJSSE and Conscrypt security providers. To choose a provider, specify either “SunJSSE” or “Conscrypt” string as an argument to this method.
<code>tunnelingTrustManagerAlgorithm(String)</code>	Optional. Specifies a Java trust manager algorithm for tunneling connection.
<code>username(const String&)</code>	Specifies the name used in the login request. Overrides a value specified in the Login domain via the addAdminMsg(const ReqMsg&) method.

Table 31: OmmNiProviderConfig Class Method Calls

4.4.2 Using the `host()` Function: How Host and Port Parameters are Processed

Host and **Port** parameters both have global default values. Thus, if either an `OmmConsumerConfig` or `OmmNiProviderConfig` object exists, its **Host** and **Port** will always have values (either the default value or some other value as specified in a configuration XML file such as `EmaConfig.xml`).

- The default **Host:Port** value for `OmmConsumerConfig` is `localhost:14002`.
- The default **Host:Port** value for `OmmNiProviderConfig` is `localhost:14003`.

If needed, you can have the application reset both host and port values by calling the `host( String )` method on the object using the syntax: **HostValue:PortValue**.

NOTE: Calling the `host()` function sets `channelType` (refer to Section 3.4.2) to `RSSL_SOCKET`, regardless of how it was previously configured.

Host and **Port** values observe the following rules when updating due to the `host( String )` method:

- If the host parameter is missing or empty, then host and port reset to their global default values.
- If the host parameter is set to the string “:”, then host and port reset to their global default values.
- If the host parameter is a string (not containing a :), then host is set to that string and port resets to its default value.
- If the parameter begins with a : and is followed by some text, then host is set to its global default value and port is set to that text.
- If the parameter is **HostValue:PortValue**, where both **HostValue** and **PortValue** have values, then host is set to **HostValue** and port is set to **PortValue**.

4.4.3 Service Discovery Configuration Using Function Calls

4.4.3.1 ServiceEndpointDiscovery

`ServiceEndpointDiscovery` class provides the functionality to query endpoints from RDP service discovery.

The application interacts with service discovery through the `ServiceEndpointDiscovery` interface methods.

The results of these interactions are communicated back to application through `ServiceEndpointDiscoveryClient`.

Please refer to the reference manual manual for more details on `ServiceEndpointDiscovery` and the classes it uses for functionality.

4.5 Helper Method Calls

4.5.1 OmmConsumer Helper Method Calls

4.5.1.1 FallBack to Preferred Host Method Call

Enterprise Message API application may trigger a fallback to preferred host in a ChannelSet configuration or fallback to a preferred group in WarmStandby configuration.

4.5.1.1.1 Method Call

When the application directly calls `OmmConsumer::fallbackPreferredHost()`, the library will immediately start a preferred host operation. If preferred host is not enabled on the channel, it returns an error. If a preferred host operation is in progress, it immediately returns success. Once the shift to preferred host channel/warm standby group completes, the library sends Login StatusMsg with dataState `RSSL_DATA_OK`, streamState `RSSL_STREAM_OPEN`, text "Preferred host complete", and code `OmmState.SocketPHComplete`.

4.5.1.2 Dynamically Set Configuration Using IOCTL: Preferred Host Options

All preferred host options can be modified during runtime via `OmmConsumer::modifyIOctl()` calls. `PreferredHostOptions` class must be used to alter preferred host configuration. If IOCTL method is called while a preferred host operation is in progress, the change will be applied after the operation completes. For details on each of the following preferred host parameters, refer to .

METHOD	DESCRIPTION
<code>enablePreferredHostOptions(bool)</code>	Specifies the value of preferred host parameter EnablePreferredHostOptions .
<code>phDetectionTimeSchedule(String)</code>	Specifies the value of preferred host parameter PHDetectionTimeSchedule .
<code>phDetectionTimeInterval(int)</code>	Specifies the value of preferred host parameter PHDetectionTimeInterval .
<code>preferredChannelName(String)</code>	Specifies the value of preferred host parameter PreferredChannelName .
<code>preferredWSBChannelName(String)</code>	Specifies name of WarmStandby group to set as preferred WarmStandby group.
<code>fallBackWithinWSBGroup(bool)</code>	Specifies value of preferred host parameter PHFallBackWithinWSBGroup .

Table 32: Preferred Host Options

4.5.1.3 GetChannelInfo Method Call: Preferred Host Parameters

Enterprise Message API application may request information regarding configured preferred host parameters. The following table describes the values contained in the `PreferredHostInfo` class. This information is available through the `OmmConsumer.getChannelInfo().getPreferredHostInfo()` method and is returned as a part of the `ChannelInformation` structure.

METHOD	DESCRIPTION
<code>getEnablePreferredHostOptions()</code>	Return the value of preferred host parameter EnablePreferredHostOptions .
<code>getPHDetectionTimeSchedule()</code>	Return the value of preferred host parameter PHDetectionTimeSchedule .
<code>getPHDetectionTimeInterval()</code>	Return the value of preferred host parameter PHDetectionTimeInterval .
<code>getPreferredChannelName()</code>	Return the value of preferred host parameter PreferredChannelName .
<code>getPreferredWSBChannelName()</code>	Return the value of preferred host parameter PreferredWSBChannelName .
<code>getPHFallBackWithinWSBGroup()</code>	Return the value of preferred host parameter PHFallBackWithinWSBGroup .
<code>getRemainingDetectionTime()</code>	Return the remaining time, in seconds, until the library performs the next preferred host operation.

Table 33: Preferred Host Parameters

4.6 Programmatic Configuration

In addition to changing the Enterprise Message API's configuration via an XML configuration file (e.g., **EmaConfig.xml**) or function calls, you can programmatically change the API's behavior via an OMM data structure.

4.6.1 OMM Data Structure

Programmatic configuration of the Enterprise Message API provides a way of configuring all parameters using an OMM data structure, which is divided into four tiers:

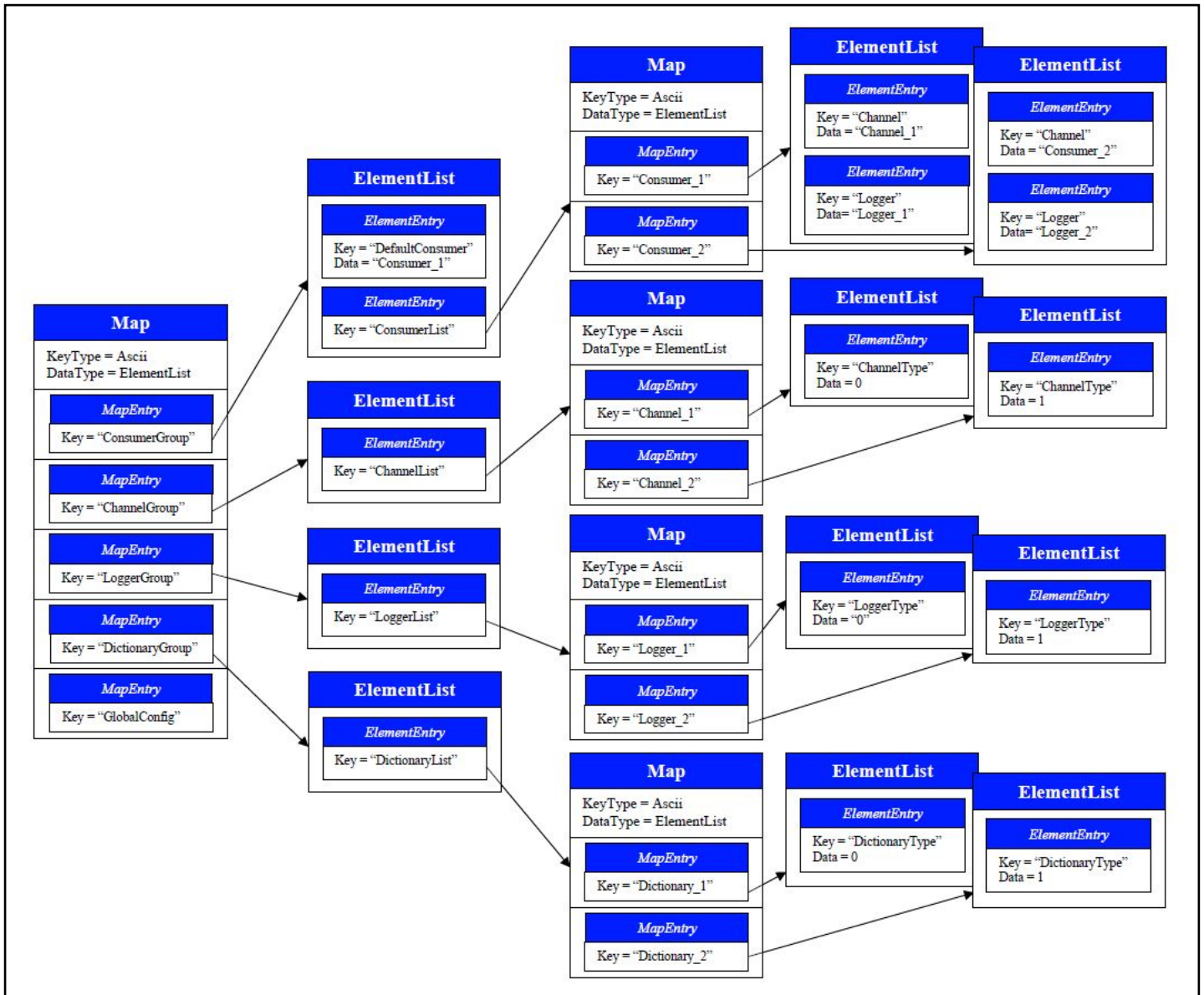
- The 1st tier lists the following Enterprise Message API's components each of which has its own list in the 2nd tier:
 - Consumer
 - IProvider
 - NiProvider
 - Channel
 - WarmStandby
 - WarmStandbyServerInfo
 - SessionChannel
 - Server
 - Dictionary
 - Directory
- The 2nd tier includes each component's list and the default consumers and providers for use when loading configuration parameters.
- The 3rd tier defines individual names for these components, which then have their own configuration parameters in 4th tier.
- The 4th tier defines configuration parameters that are assigned to specific components.

4.6.2 Creating a Programmatic Configuration for a Consumer

NOTE: When encoding OMM types, you must follow the OMM data structure and configuration parameter types listed in this document.

► To programmatically configure an Enterprise Message API Consumer:

1. Create a map with the following hierarchy to configure Enterprise Message API configuration parameters:



2. Call the **config** method on an **OmmConsumerConfig** object, and pass the Map (which represents the programmatic OMM structure) as a parameter to the **config** method.

You can pass in multiple maps, each programmatic configuration being applied to create the application's active configuration during instantiation of the **OmmConsumer** or **OmmProvider**.

4.6.3 Example: Programmatic Configuration of the Consumer

The following example illustrates programmatically configuring a consumer:

```
Map innerMap = EmaFactory.createMap();
Map configMap = EmaFactory.createMap();
ElementList elementList = EmaFactory.createElementList();
ElementList innerElementList = EmaFactory.createElementList();

innerElementList.add(EmaFactory.createElementEntry().intValue("ReactorMsgEventPoolLimit", 2000));
innerElementList.add(EmaFactory.createElementEntry().intValue("ReactorChannelEventPoolLimit", 1500));
innerElementList.add(EmaFactory.createElementEntry().intValue("WorkerEventPoolLimit", 1000));
innerElementList.add(EmaFactory.createElementEntry().intValue("TunnelStreamMsgEventPoolLimit",
    2500));
innerElementList.add(EmaFactory.createElementEntry().intValue("TunnelStreamStatusEventPoolLimit",
    3000));
configMap.add(EmaFactory.createMapEntry().keyAscii( "GlobalConfig", MapEntry.MapAction.ADD,
    innerElementList ));
innerElementList.clear();

elementList.add(EmaFactory.createElementEntry().ascii("DefaultConsumer", "Consumer_1"));
innerElementList.add(EmaFactory.createElementEntry().ascii("ChannelSet", "Channel_1, Channel_2"));
innerElementList.add(EmaFactory.createElementEntry().ascii("Dictionary", "Dictionary_1"));
innerMap.add(EmaFactory.createMapEntry().keyAscii( "Consumer_1", MapEntry.MapAction.ADD,
    innerElementList));
innerElementList.clear();

elementList.add(EmaFactory.createElementEntry().map( "ConsumerList", innerMap ));
innerMap.clear();
configMap.add(EmaFactory.createMapEntry().keyAscii( "ConsumerGroup", MapEntry.MapAction.ADD,
    elementList ));
elementList.clear();

innerElementList.add(EmaFactory.createElementEntry().ascii("ChannelType",
    "ChannelType::RSSL_SOCKET"));
innerElementList.add(EmaFactory.createElementEntry().ascii("Host", "localhost"));
innerElementList.add(EmaFactory.createElementEntry().ascii("Port", "14002"));
innerMap.add(EmaFactory.createMapEntry().keyAscii( "Channel_1", MapEntry.MapAction.ADD,
    innerElementList));
innerElementList.clear();

innerElementList.add(EmaFactory.createElementEntry().ascii("ChannelType",
    "ChannelType::RSSL_SOCKET"));
innerElementList.add(EmaFactory.createElementEntry().ascii("Host", "121.1.1.100"));
innerElementList.add(EmaFactory.createElementEntry().ascii("Port", "14008"));
innerMap.add(EmaFactory.createMapEntry().keyAscii( "Channel_2", MapEntry.MapAction.ADD,
    innerElementList));
innerElementList.clear();

elementList.add(EmaFactory.createElementEntry().map( "ChannelList", innerMap ));
```

```

innerMap.clear();

configMap.add(EmaFactory.createMapEntry().keyAscii( "ChannelGroup", MapEntry.MapAction.ADD,
    elementList ));
elementList.clear();

innerElementList.add(EmaFactory.createElementEntry().ascii("DictionaryType",
    "DictionaryType::ChannelDictionary"));
innerElementList.add(EmaFactory.createElementEntry().ascii("RdmFieldDictionaryFileName", "./
    RDMFieldDictionary"));
innerElementList.add(EmaFactory.createElementEntry().ascii("EnumTypeDefFileName", "./enumtype.def"));
innerMap.add(EmaFactory.createMapEntry().keyAscii( "Dictionary_1", MapEntry.MapAction.ADD,
    innerElementList));
innerElementList.clear();

elementList.add(EmaFactory.createElementEntry().map( "DictionaryList", innerMap ));
configMap.add(EmaFactory.createMapEntry().keyAscii( "DictionaryGroup", MapEntry.MapAction.ADD,
    elementList ));
elementList.clear();

...

consumer = EmaFactory.createOmmConsumer(EmaFactory.createOmmConsumerConfig().config(configMap));

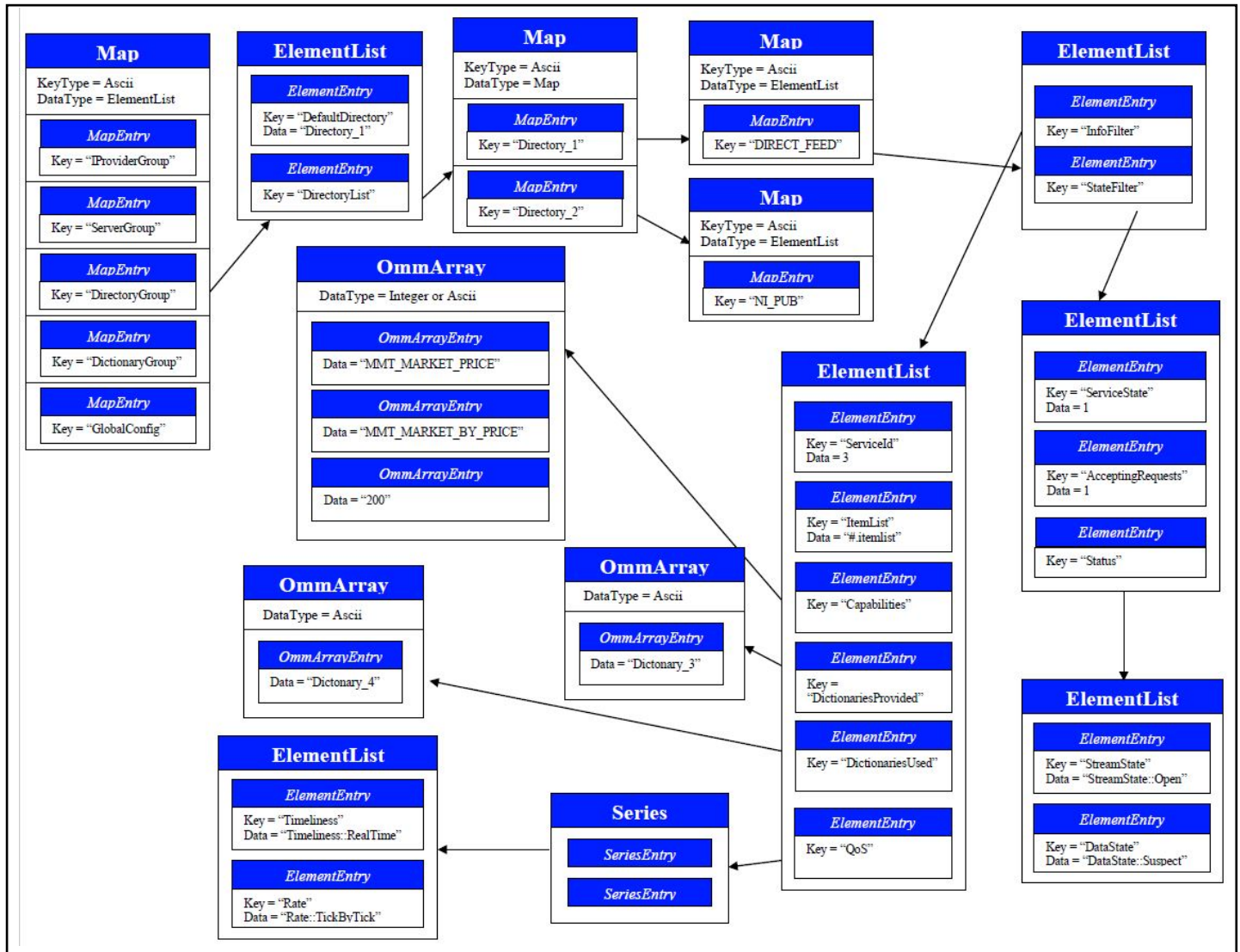
```

4.6.4 Creating a Programmatic Configuration for a Provider

NOTE: When encoding OMM types, you must follow the OMM data structure and configuration parameter types listed in this document.

► To programmatically configure an Enterprise Message API IProvider:

1. To configure an Enterprise Message API directory's configuration parameters, create a map with the following hierarchy:



2. Call the **config** method on an **OmmIProviderConfig** object, and pass the Map (which represents the programmatic OMM structure) as a parameter to the **config** method.

You can pass in multiple maps, each programmatic configuration being applied to create the application's active configuration during instantiation of the **OmmConsumer** or **OmmProvider**.

NOTE: You must set **adminControlDirectory** and **adminControlDictionary** to their default settings (**ApiControlEnum**) when programmatically configuring:

- A Directory Refresh message published by an IProvider, or
- A Dictionary Refresh message published by an IProvider

4.6.5 Example: Programmatic Configuration of a Provider

The following example illustrates programmatically configuring a Provider:

```
Map outermostMap = EmaFactory.createMap();
Map innerMap = EmaFactory.createMap();
ElementList elementList = EmaFactory.createElementList();
ElementList innerElementList = EmaFactory.createElementList();

innerElementList.add(EmaFactory.createElementEntry().intValue("ReactorMsgEventPoolLimit", 2000));
innerElementList.add(EmaFactory.createElementEntry().intValue("ReactorChannelEventPoolLimit", 1500));
innerElementList.add(EmaFactory.createElementEntry().intValue("WorkerEventPoolLimit", 1000));
innerElementList.add(EmaFactory.createElementEntry().intValue("TunnelStreamMsgEventPoolLimit",
    2500));
innerElementList.add(EmaFactory.createElementEntry().intValue("TunnelStreamStatusEventPoolLimit",
    3000));
outermostMap.add(EmaFactory.createMapEntry().keyAscii( "GlobalConfig", MapEntry.MapAction.ADD,
    innerElementList ));
innerElementList.clear();

elementList.add(EmaFactory.createElementEntry().ascii("DefaultIPProvider", "Provider_1"));

innerElementList.add(EmaFactory.createElementEntry().ascii("Server", "Server_1"));
innerElementList.add(EmaFactory.createElementEntry().ascii("Directory", "Directory_1"));

innerElementList.add(EmaFactory.createElementEntry().intValue("ItemCountHint", 5000));
innerElementList.add(EmaFactory.createElementEntry().intValue("ServiceCountHint", 5000));
innerElementList.add(EmaFactory.createElementEntry().intValue("AcceptDirMessageWithoutMinFilters",
    0));
innerElementList.add(EmaFactory.createElementEntry().intValue("AcceptMessageSameKeyButDiffStream",
    0));
innerElementList.add(EmaFactory.createElementEntry().intValue("RefreshFirstRequired", 1));
innerElementList.add(EmaFactory.createElementEntry().intValue("RequestTimeout", 5000));
innerElementList.add(EmaFactory.createElementEntry().intValue("DispatchTimeoutApiThread", 5656));
innerElementList.add(EmaFactory.createElementEntry().intValue("MaxDispatchCountApiThread", 500));
innerElementList.add(EmaFactory.createElementEntry().intValue("MaxDispatchCountUserThread", 500));
innerElementList.add(EmaFactory.createElementEntry().intValue("XmlTraceToStdout", 0));

innerMap.add(EmaFactory.createMapEntry().keyAscii( "Provider_1", MapEntry.MapAction.ADD,
    innerElementList));
innerElementList.clear();

elementList.add(EmaFactory.createElementEntry().map("IPProviderList", innerMap));
innerMap.clear();

outermostMap.add(EmaFactory.createMapEntry().keyAscii( "IPProviderGroup", MapEntry.MapAction.ADD,
    elementList));
elementList.clear();
```

```

innerElementList.add(EmaFactory.createElementEntry().ascii("ServerType", "ServerType::RSSL_SOCKET"));
innerElementList.add(EmaFactory.createElementEntry().ascii("CompressionType",
    "CompressionType::LZ4"));
innerElementList.add(EmaFactory.createElementEntry().intValue("GuaranteedOutputBuffers", 7000));
innerElementList.add(EmaFactory.createElementEntry().intValue("NumInputBuffers", 5000));
innerElementList.add(EmaFactory.createElementEntry().intValue("ConnectionPingTimeout", 70000));
innerElementList.add(EmaFactory.createElementEntry().ascii("Port", "14003"));
innerElementList.add(EmaFactory.createElementEntry().intValue("TcpNoDelay", 1));
innerMap.add(EmaFactory.createMapEntry().keyAscii( "Server_1", MapEntry.MapAction.ADD,
    innerElementList));
innerElementList.clear();

elementList.add(EmaFactory.createElementEntry().map("ServerList", innerMap));
innerMap.clear();

outermostMap.add(EmaFactory.createMapEntry().keyAscii( "ServerGroup", MapEntry.MapAction.ADD,
    elementList));
elementList.clear();

innerElementList.add(EmaFactory.createElementEntry().ascii("DictionaryType",
    "DictionaryType::FileDictionary"));
innerElementList.add(EmaFactory.createElementEntry().ascii("RdmFieldDictionaryItemName", "RWFFld"));
innerElementList.add(EmaFactory.createElementEntry().ascii("EnumTypeDefItemName", "RWFEEnum"));
innerElementList.add(EmaFactory.createElementEntry().ascii("RdmFieldDictionaryFileName", "./
    RDMFieldDictionary"));
innerElementList.add(EmaFactory.createElementEntry().ascii("EnumTypeDefFileName", "./enumtype.def"));
innerMap.add(EmaFactory.createMapEntry().keyAscii( "Dictionary_1", MapEntry.MapAction.ADD,
    innerElementList));
innerElementList.clear();

elementList.add(EmaFactory.createElementEntry().map("DictionaryList", innerMap));
innerMap.clear();

outermostMap.add(EmaFactory.createMapEntry().keyAscii( "DictionaryGroup", MapEntry.MapAction.ADD,
    elementList));
elementList.clear();

Map serviceMap = EmaFactory.createMap();
ElementList infoElementList = EmaFactory.createElementList();
ElementList stateElementList = EmaFactory.createElementList();
OmmArray infoArray = EmaFactory.createOmmArray();
Series qosSeries = EmaFactory.createSeries();

infoElementList.add(EmaFactory.createElementEntry().intValue("ServiceId", 1));
infoElementList.add(EmaFactory.createElementEntry().ascii("Vendor", "Vendor"));
infoElementList.add(EmaFactory.createElementEntry().intValue("IsSource", 1));
infoElementList.add(EmaFactory.createElementEntry().intValue("AcceptingConsumerStatus", 1));
infoElementList.add(EmaFactory.createElementEntry().intValue("SupportsQoSRange", 1));
infoElementList.add(EmaFactory.createElementEntry().intValue("SupportsOutOfBandSnapshots", 1));
infoElementList.add(EmaFactory.createElementEntry().ascii("ItemList", "#.itemlist"));

```

```

infoArray.add(EmaFactory.createOmmArrayEntry().ascii("MMT_MARKET_PRICE"));
infoArray.add(EmaFactory.createOmmArrayEntry().ascii("MMT_MARKET_BY_PRICE"));
infoArray.add(EmaFactory.createOmmArrayEntry().ascii("MMT_MARKET_BY_ORDER"));
infoArray.add(EmaFactory.createOmmArrayEntry().ascii("130"));
infoElementList.add(EmaFactory.createElementEntry().array("Capabilities", infoArray));
infoArray.clear();

infoArray.add(EmaFactory.createOmmArrayEntry().ascii("Dictionary_1"));
infoElementList.add(EmaFactory.createElementEntry().array("DictionariesProvided", infoArray));
infoArray.clear();
infoArray.add(EmaFactory.createOmmArrayEntry().ascii("Dictionary_1"));
infoElementList.add(EmaFactory.createElementEntry().array("DictionariesUsed", infoArray));
infoArray.clear();

innerElementList.add(EmaFactory.createElementEntry().ascii("Timeliness", "Timeliness::RealTime"));
innerElementList.add(EmaFactory.createElementEntry().ascii("Rate", "Rate::TickByTick"));
qosSeries.add(EmaFactory.createSeriesEntry().elementList(innerElementList));
innerElementList.clear();

infoElementList.add(EmaFactory.createElementEntry().series("QoS", qosSeries));
qosSeries.clear();

stateElementList.add(EmaFactory.createElementEntry().intValue("ServiceState", 1));
stateElementList.add(EmaFactory.createElementEntry().intValue("AcceptingRequests", 1));

innerElementList.add(EmaFactory.createElementEntry().ascii("StreamState", "StreamState::Open"));
innerElementList.add(EmaFactory.createElementEntry().ascii("DataState", "DataState::Suspect"));
innerElementList.add(EmaFactory.createElementEntry().ascii("StatusCode", "StatusCode::DacsDown"));
innerElementList.add(EmaFactory.createElementEntry().ascii("StatusText", "dacsDown"));

stateElementList.add(EmaFactory.createElementEntry().elementList("Status", innerElementList));
innerElementList.clear();

innerElementList.add(EmaFactory.createElementEntry().elementList("InfoFilter", infoElementList));
infoElementList.clear();
innerElementList.add(EmaFactory.createElementEntry().elementList("StateFilter", stateElementList));
stateElementList.clear();

serviceMap.add(EmaFactory.createMapEntry().keyAscii( "DIRECT_FEED", MapEntry.MapAction.ADD,
    innerElementList));
innerElementList.clear();
innerMap.add(EmaFactory.createMapEntry().keyAscii( "Directory_1", MapEntry.MapAction.ADD,
    serviceMap));
serviceMap.clear();

elementList.add(EmaFactory.createElementEntry().map("DirectoryList", innerMap));
innerMap.clear();

outermostMap.add(EmaFactory.createMapEntry().keyAscii( "DirectoryGroup", MapEntry.MapAction.ADD,
    elementList));

```

```
provider = EmaFactory.createOmmProvider(  
    EmaFactory.createOmmIProviderConfig().config(outermostMap).operationModel(  
        OmmIProviderConfig.OperationModel.USER_DISPATCH ), appClient );
```

© LSEG 2016 - 2026. All rights reserved.

Republication or redistribution of LSEG Data & Analytics content, including by framing or similar means, is prohibited without the prior written consent of LSEG Data & Analytics. 'LSEG Data & Analytics' and the LSEG Data & Analytics logo are registered trademarks and trademarks of LSEG Data & Analytics.

Any third party names or marks are the trademarks or registered trademarks of the relevant third party.

Document ID: EMAJ3101L1CG.260

Date of issue: September 2026



LSEG DATA &
ANALYTICS